L3 informatique

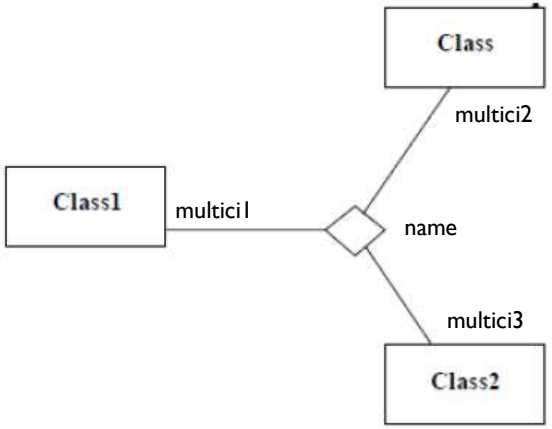
Course Notes
Software Engineering

Course 3
Modeling
Object and Class Diagram
(the continuation)

Presented by
Mr KHELIFA N.

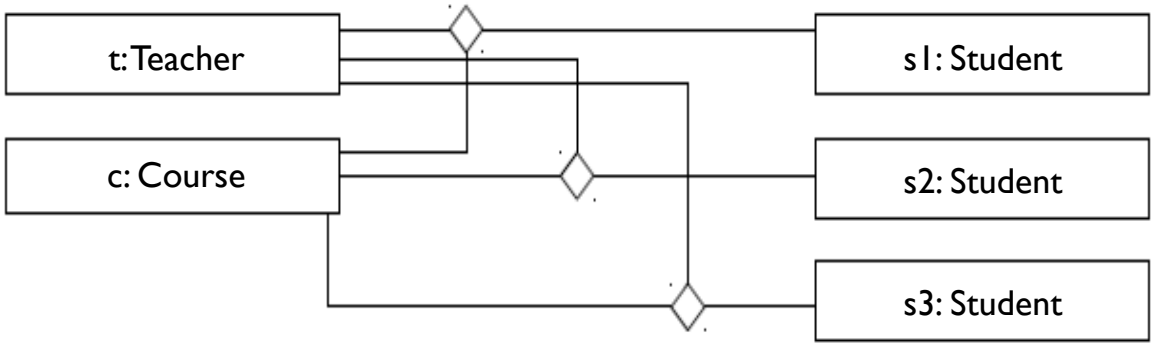
N-ary Association

- An **n-ary association** links more than two classes.
- An n-ary association is represented by a large **diamond** with a line extending to each participating class.
- The name of the association, if any, appears near the diamond.

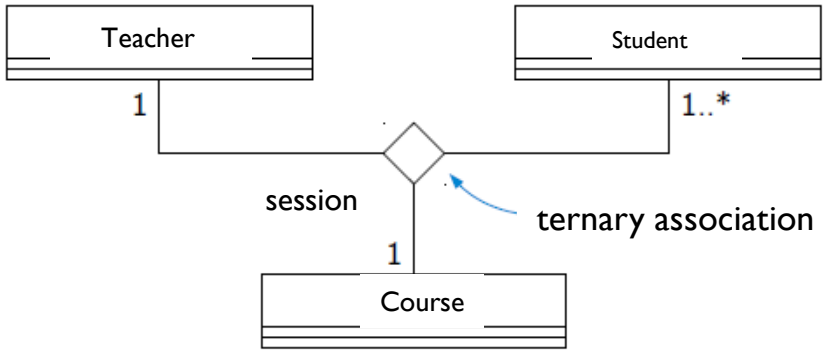


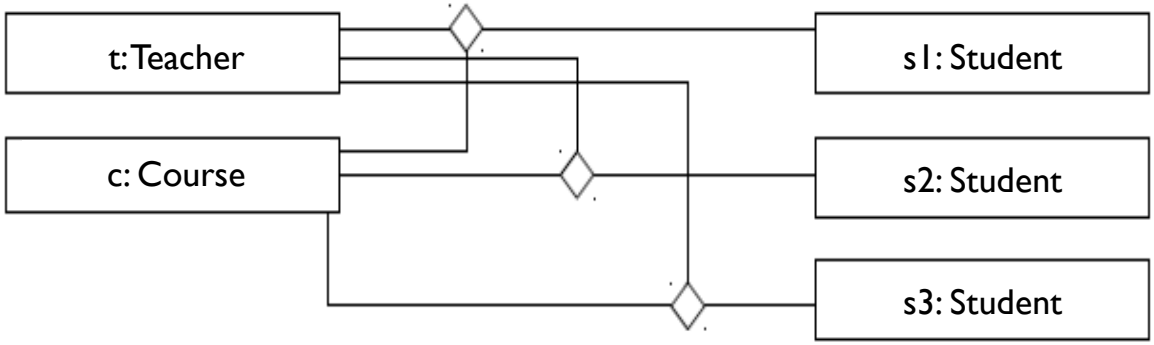
Exemple 1

- A class session may correspond to a teacher, a subject, and several students.



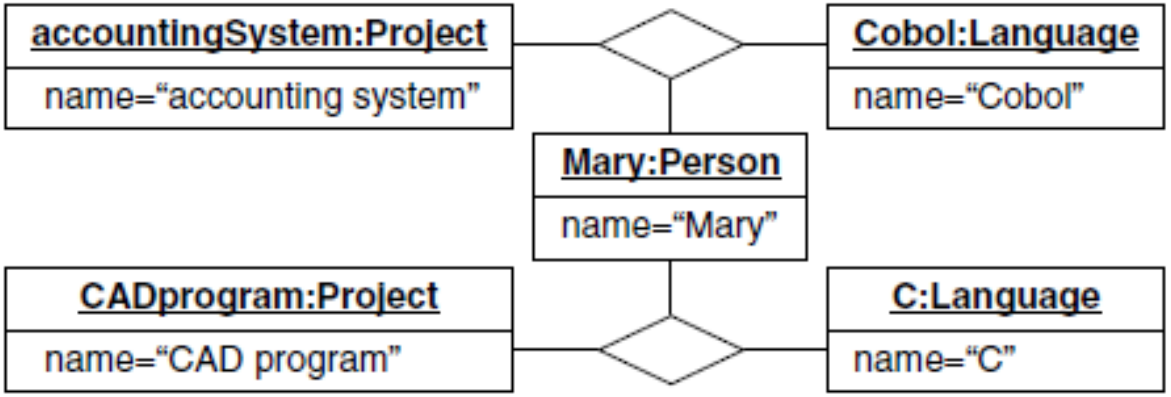
Exemple 1


Teacher
Student
1
1..*
session
ternary association
1
Course

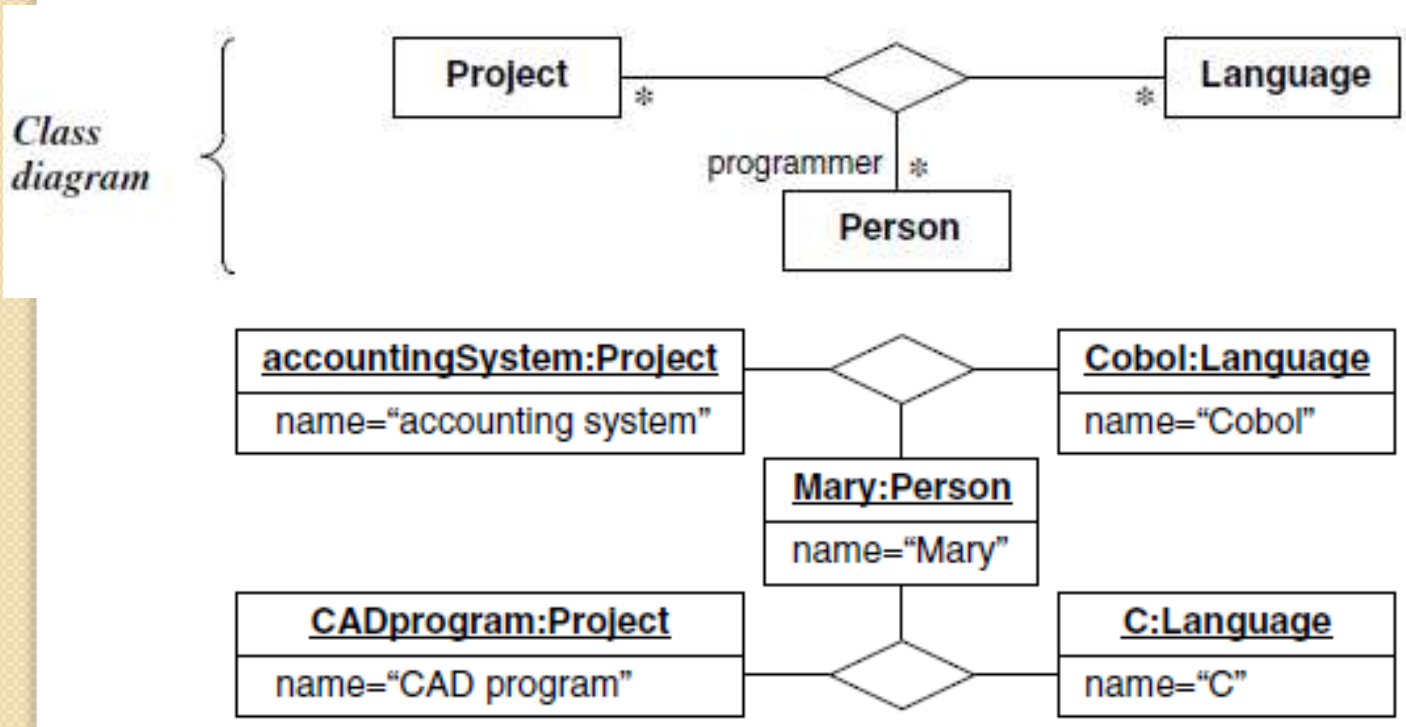

t: Teacher
c: Course
s1: Student
s2: Student
s3: Student

Exemple 2

A realization is an association of projects, programmers, and programming languages.

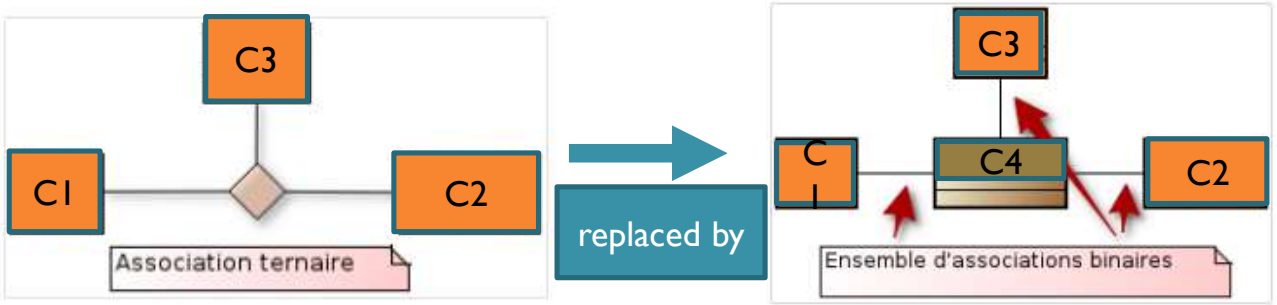


Exemple 2


Class diagram
Project
Language
programmer
Person
accountingSystem:Project
name="accounting system"
Cobol:Language
name="Cobol"
Mary:Person
name="Mary"
CADprogram:Project
name="CAD program"
C:Language
name="C"

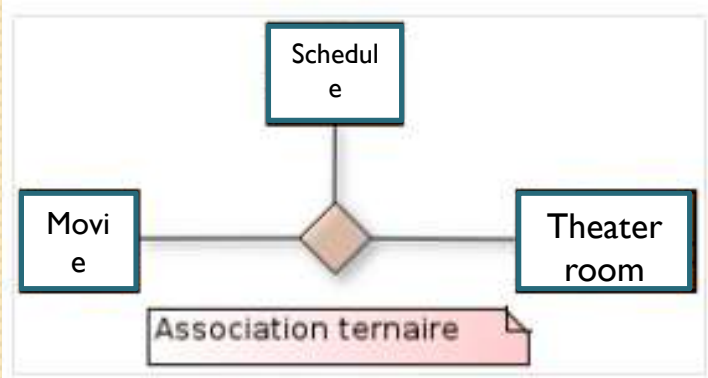
Note

- The n-ary association is imprecise, difficult to interpret, and often a source of errors; therefore, it is rarely used.
- Most of the time, it is only used to sketch the modeling at the beginning of the project, and it is soon replaced by a set of binary associations in order to remove any ambiguity.



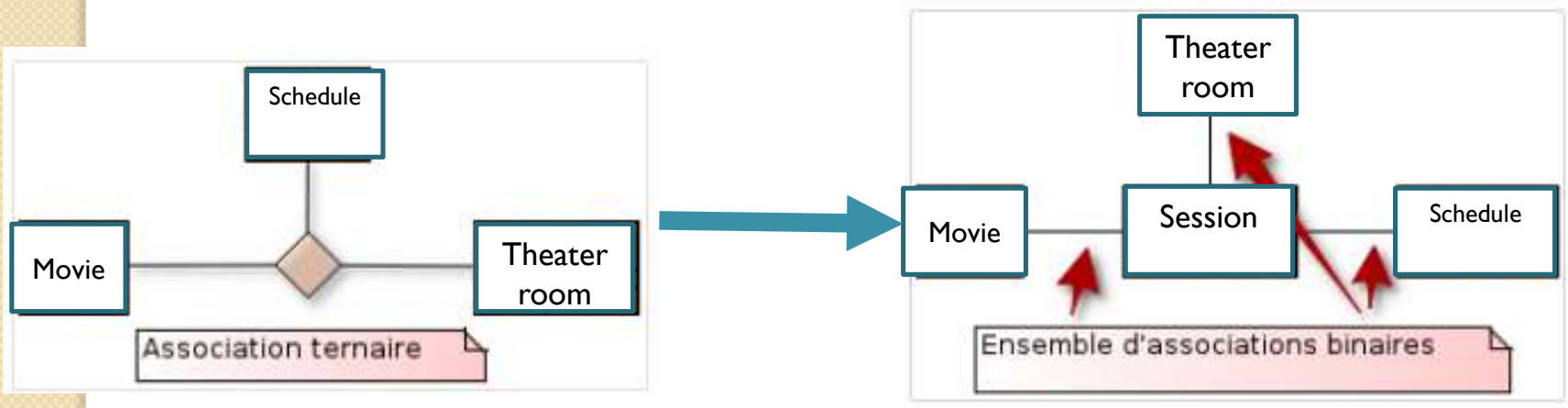
Exemple

- A movie screening may correspond to the ternary association of three classes.

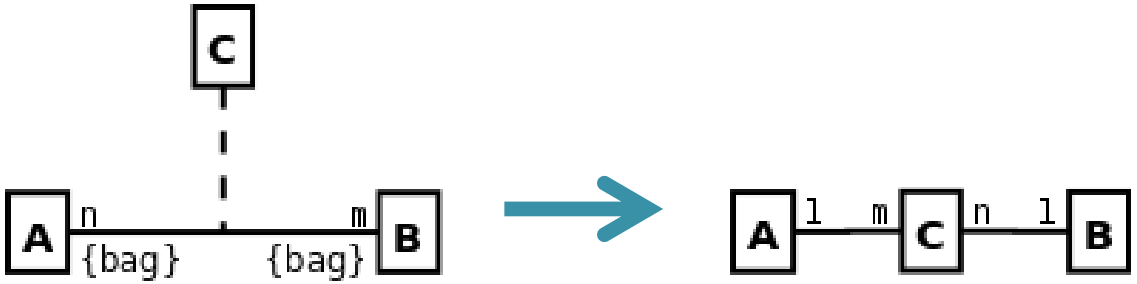


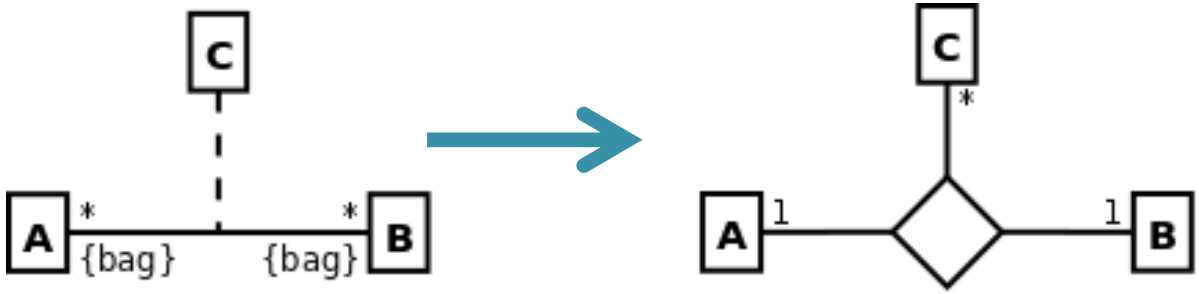
Exemple

- A movie screening may correspond to the ternary association of three classes.

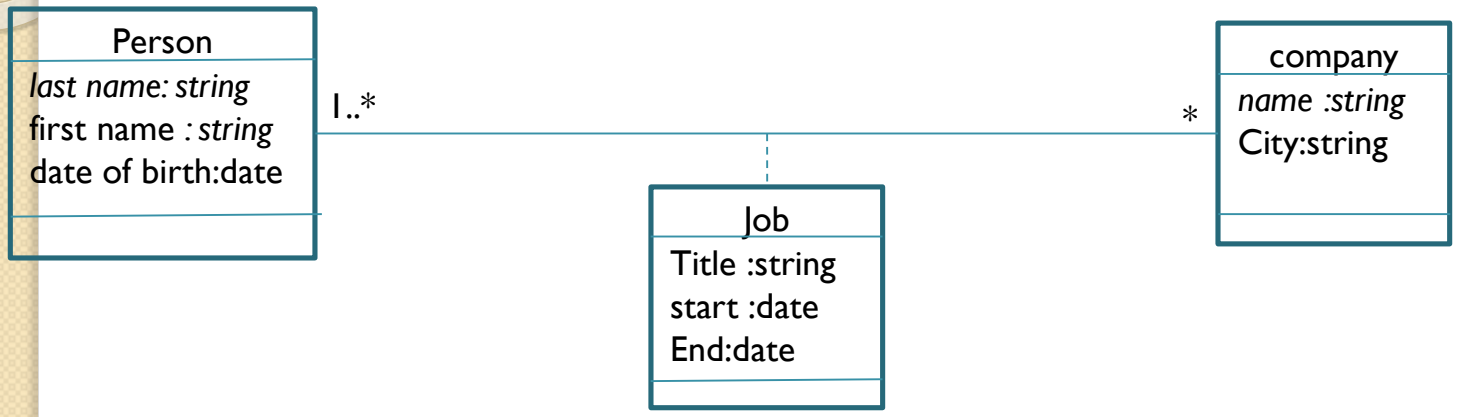


Équivalences



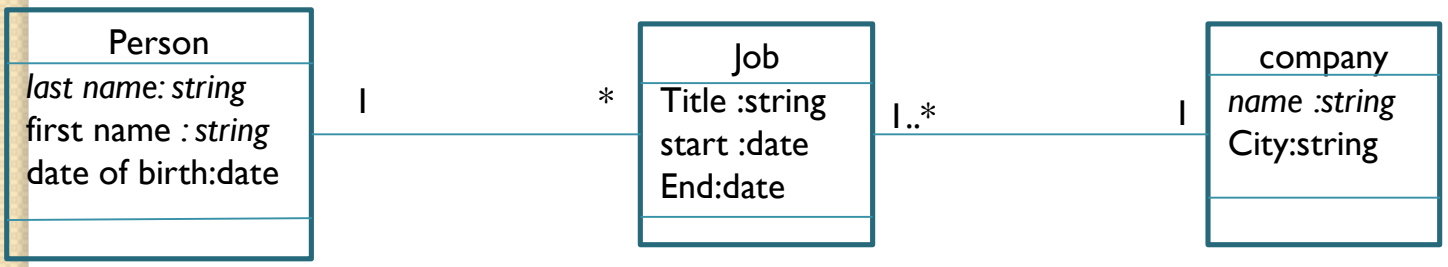


Exepmle

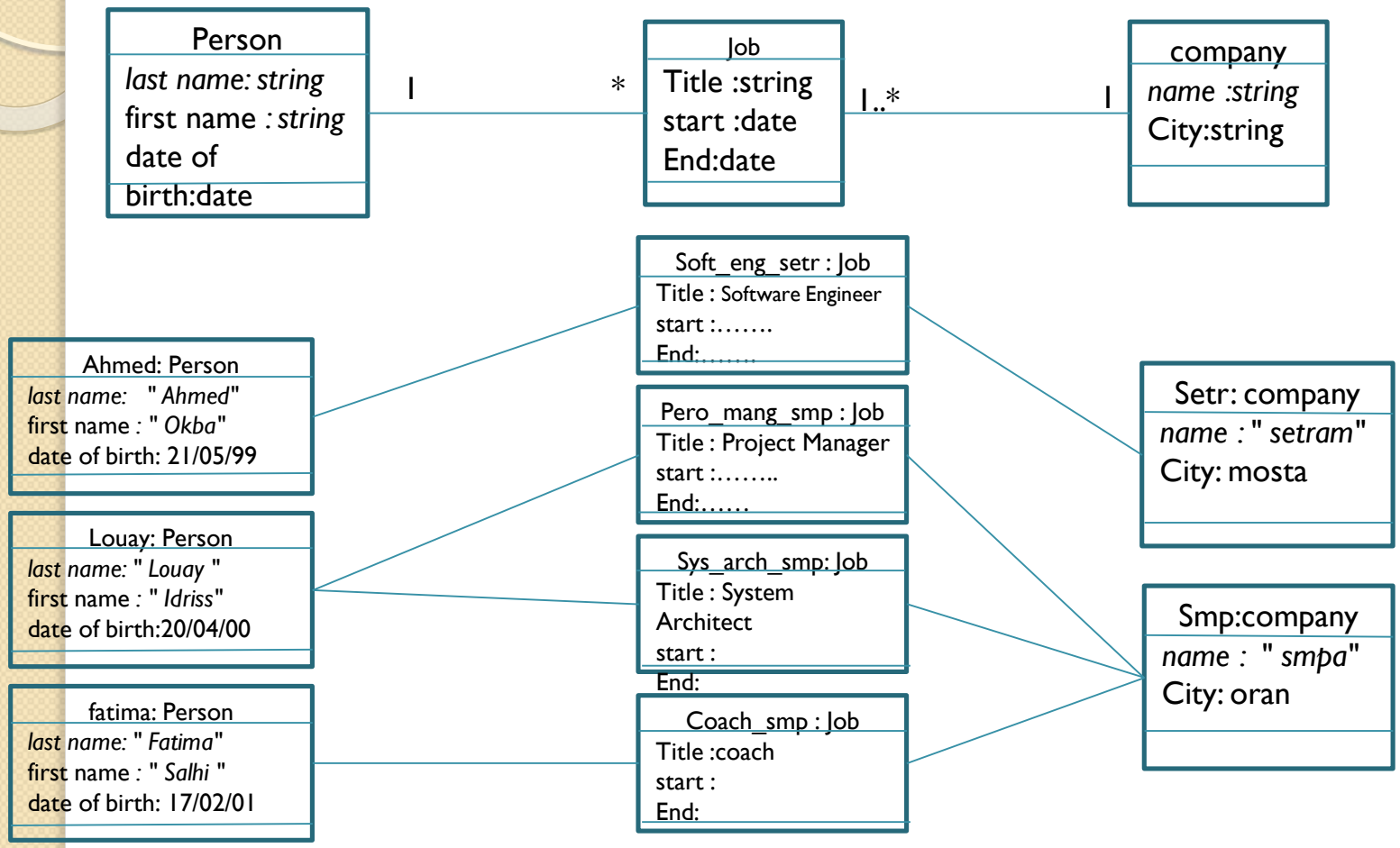


A unique instance of the association class for each link between objects.

equivalence in terms of classes and associations :

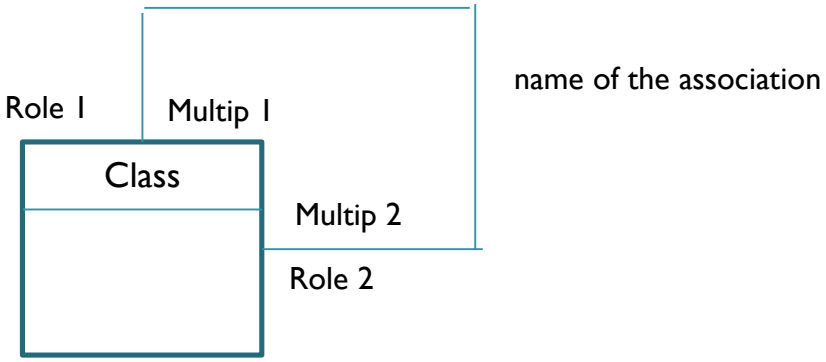


Exepmle


Person
last name: string
first name : string
date of
birth:date
1
*
Job
Title :string
start :date
End:date
1..*
1
company
name :string
City:string
Soft_eng_setr : Job
Title : Software Engineer
start :…….
End:…….
Pero_mang_smp : Job
Title : Project Manager
start :……..
End:……
Sys_arch_smp: Job
Title : System
Architect
start :
End:
Coach_smp : Job
Title :coach
start :
End:
Ahmed: Person
last name: " Ahmed"
first name : " Okba"
date of birth: 21/05/99
Louay: Person
last name: " Louay "
first name : " Idriss"
date of birth:20/04/00
fatima: Person
last name: " Fatima"
first name : " Salhi "
date of birth: 17/02/01
Setr: company
name : " setram"
City: mosta
Smp:company
name : " smpa"
City: oran

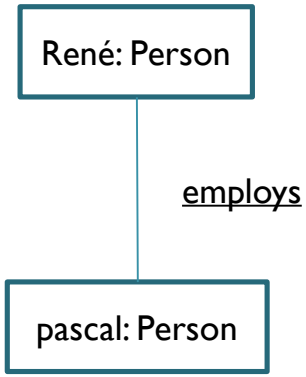
Reflexive association

- An instance of the class involved can be linked to itself or to other instances of the same class.

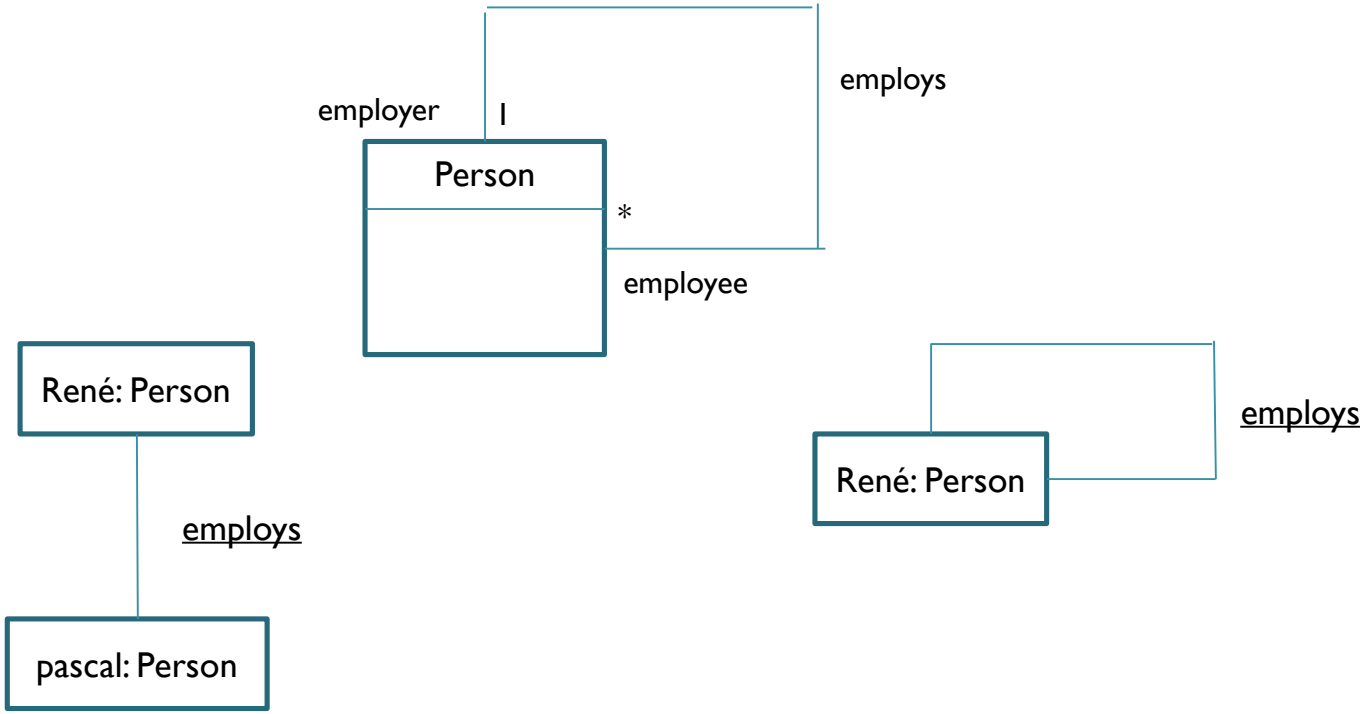


Exemple

- A person is the employer of zero or several employees, and a person is employed by an employer.

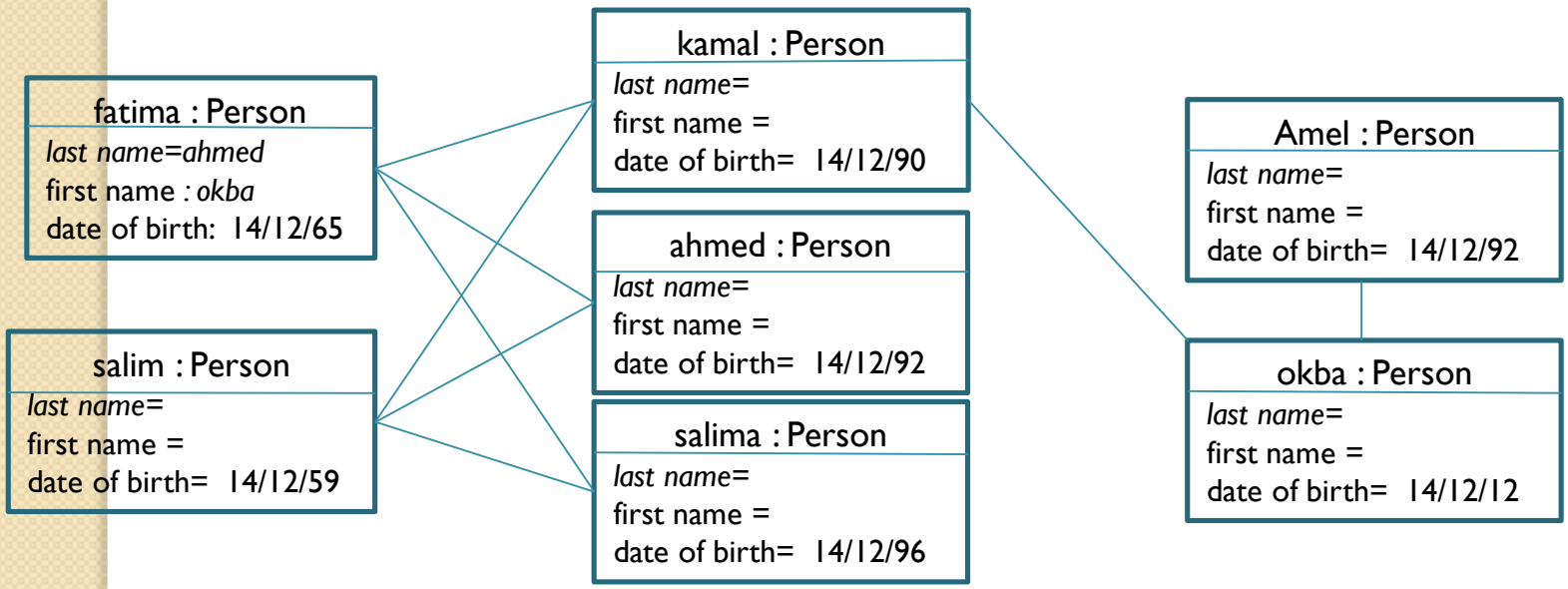


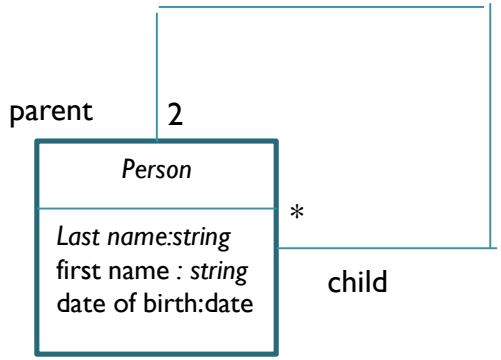
Exemple

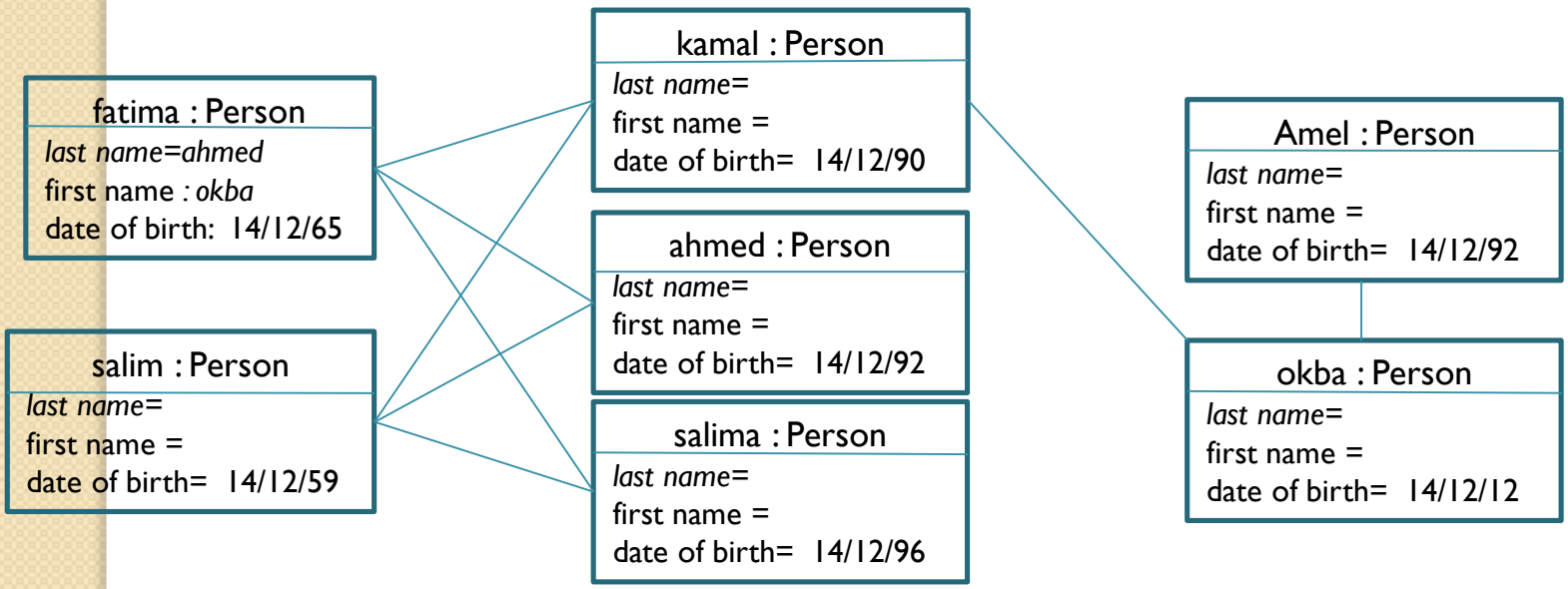

employs
employer
1
Person
*
employee
René: Person
employs
pascal: Person
employs
René: Person

Exemple 2

- A person has two parents, and a person can have zero or several children.

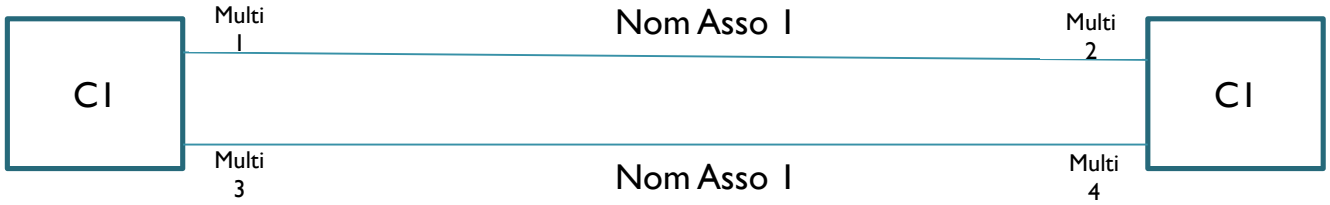



parent
2
Person
Last name:string
first name : string
date of birth:date
*
child


fatima : Person
last name=ahmed
first name : okba
date of birth: 14/12/65
salim : Person
last name=
first name =
date of birth= 14/12/59
kamal : Person
last name=
first name =
date of birth= 14/12/90
ahmed : Person
last name=
first name =
date of birth= 14/12/92
salima : Person
last name=
first name =
date of birth= 14/12/96
Amel : Person
last name=
first name =
date of birth= 14/12/92
okba : Person
last name=
first name =
date of birth= 14/12/12

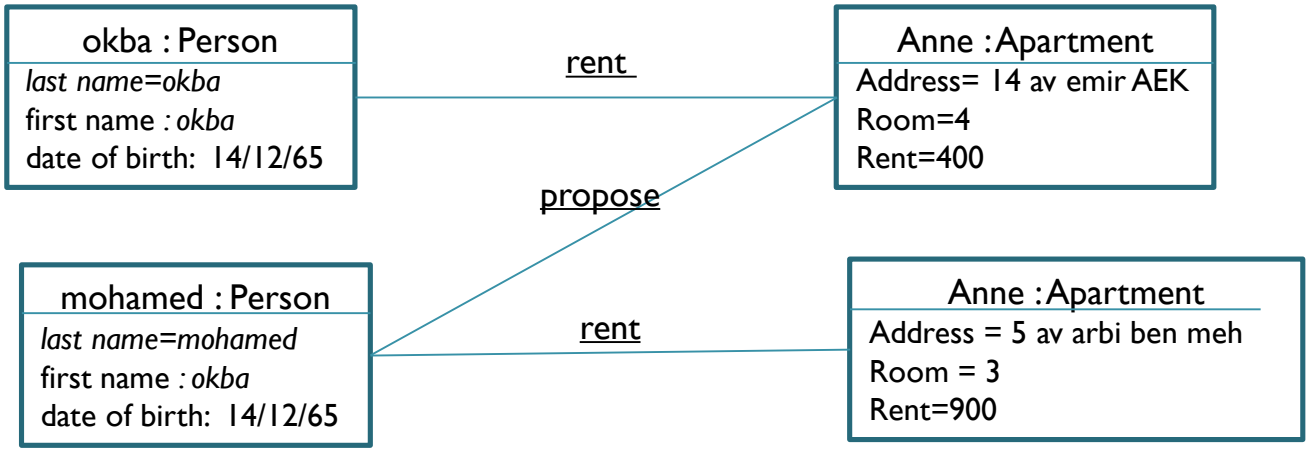
Multiple association

You can use multiple associations to model multiple links between the same objects.

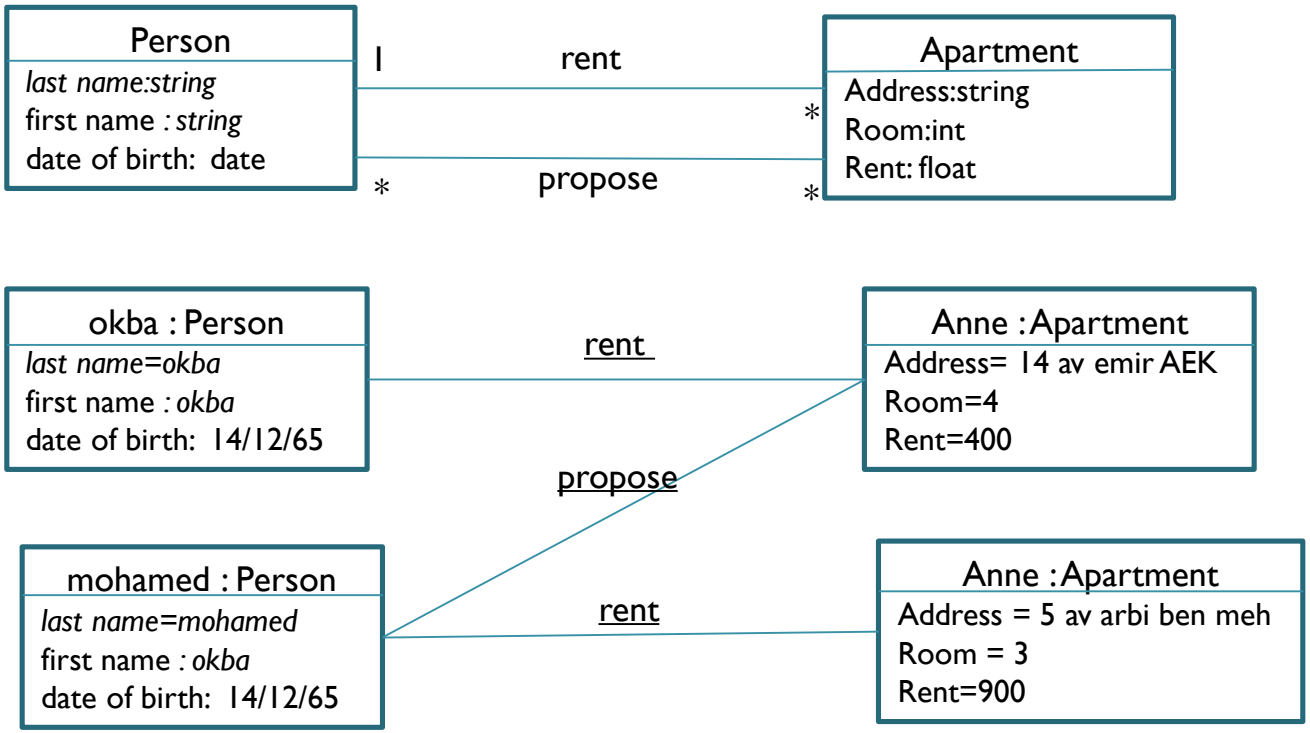


Multiple association

- A person can offer or rent zero or several apartments.
- An apartment is offered by one person and can be rented by several people.

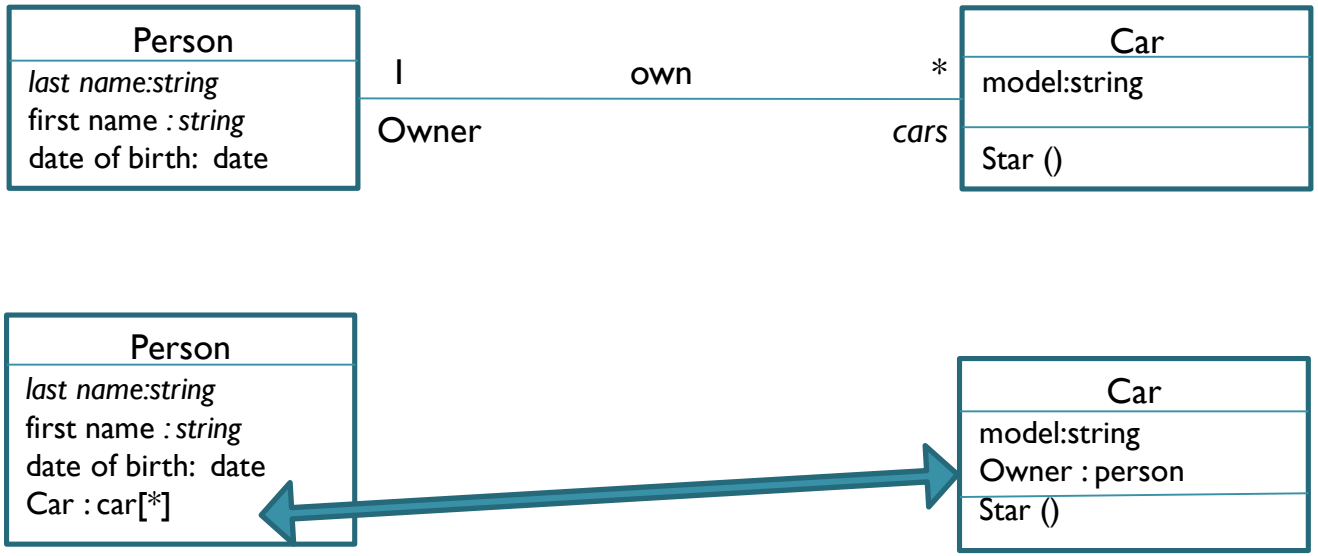


Multiple association



association

Another way to model an association

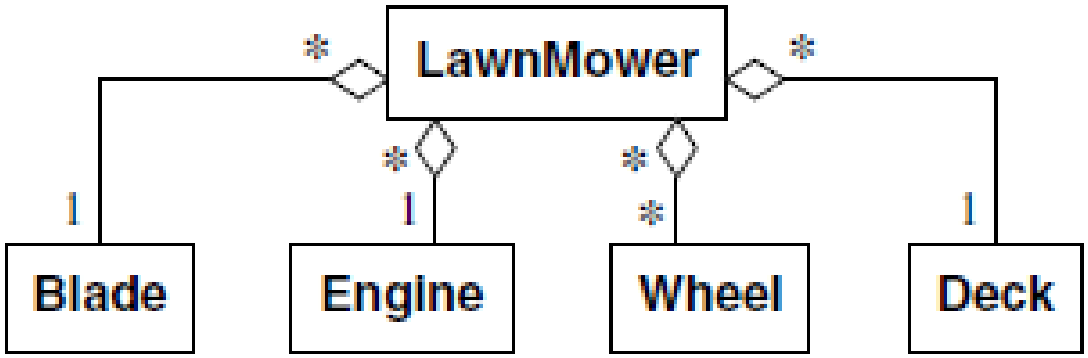


Aggregation

- **An aggregation is a special case of a non-symmetrical association that expresses a containment relationship between an element and a whole (whole/part).**
- Aggregations do not need to be named: implicitly, they mean *"contains"* or *"is composed of."*
- Aggregation is represented by adding an empty diamond on the side of the aggregate (the whole).

Exemple

- lawn mower consists of one blade, one engine, many wheels, and one deck.



Exemple

- **A felt-tip pen contains zero or one cap.**

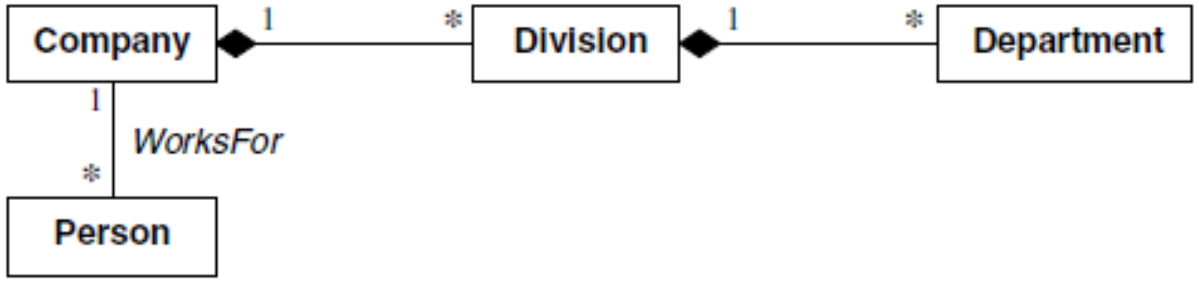
- **Note:** A felt-tip pen may lose its cap, and a cap may lose the body of its original pen. There is not necessarily consistency between the life cycles of pens and caps.

Composition

- **A composition is a stronger form of aggregation implying that:**
  - An element can belong to only one composite aggregate (non-shared aggregation).
  - The destruction of the composite aggregate (the whole) leads to the destruction of all its elements (the parts).
- The composite is responsible for the life cycle of its parts.

Exemple

- A person works for a company.
  A company employs several people.
  A company is composed of several divisions.
  A division is composed of several departments (or services).



Exemple

- A book is composed of a set of chapters.
  A chapter is composed of a set of paragraphs.

constraint

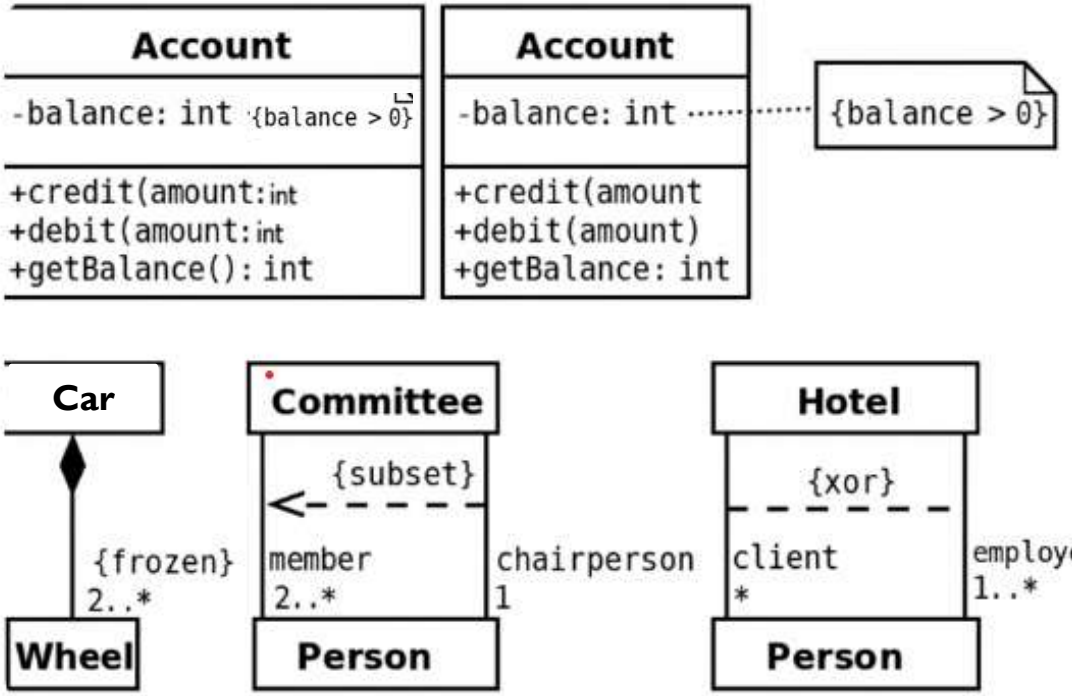
- **Properties on Association Ends**
  - **{variable}**: modifiable instance (by default)
  - **{frozen}**: non-modifiable instance
  - **{addOnly}**: instances can be added but not removed (if multiplicity > 1)

- **Predefined Constraints**
  - **On one end:** {ordered}, {unique}, ...
  - **Between two associations:** {subset}, {xor}, ...

Explanation (UMcontext):

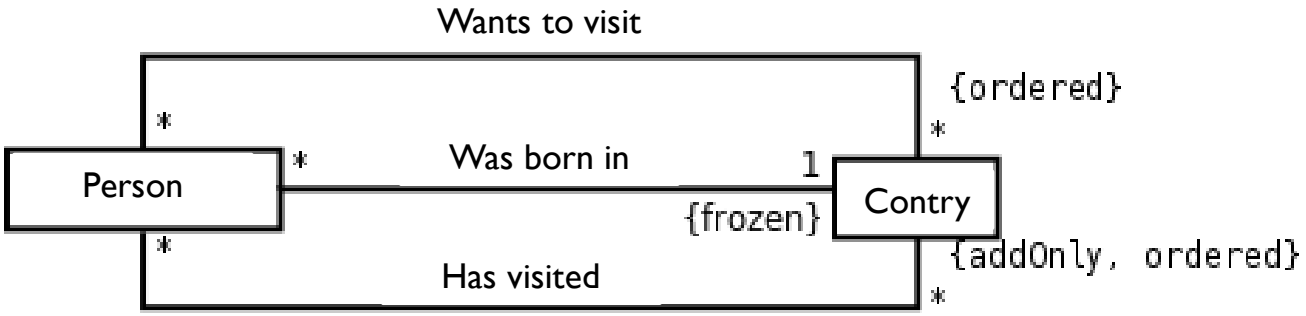
- {ordered} → the elements are kept in a specific order.
- {unique} → no duplicate elements are allowed.
- {subset} → one association end is a subset of another.
- {xor} → only one of several associations can exist at a time (exclusive relationship).

Exemple 1


Account
-balance: int
{balance > 0}
+credit(amount:int
+debit(amount:int
+getBalance(): int
Account
-balance: int
{balance > 0}
+credit(amount
+debit(amount)
+getBalance: int
Car
{frozen}
2..*
Wheel
Committee
{subset}
member
2..*
Person
chairperson
1
Hotel
{xor}
client
*
Person
employe
1..*

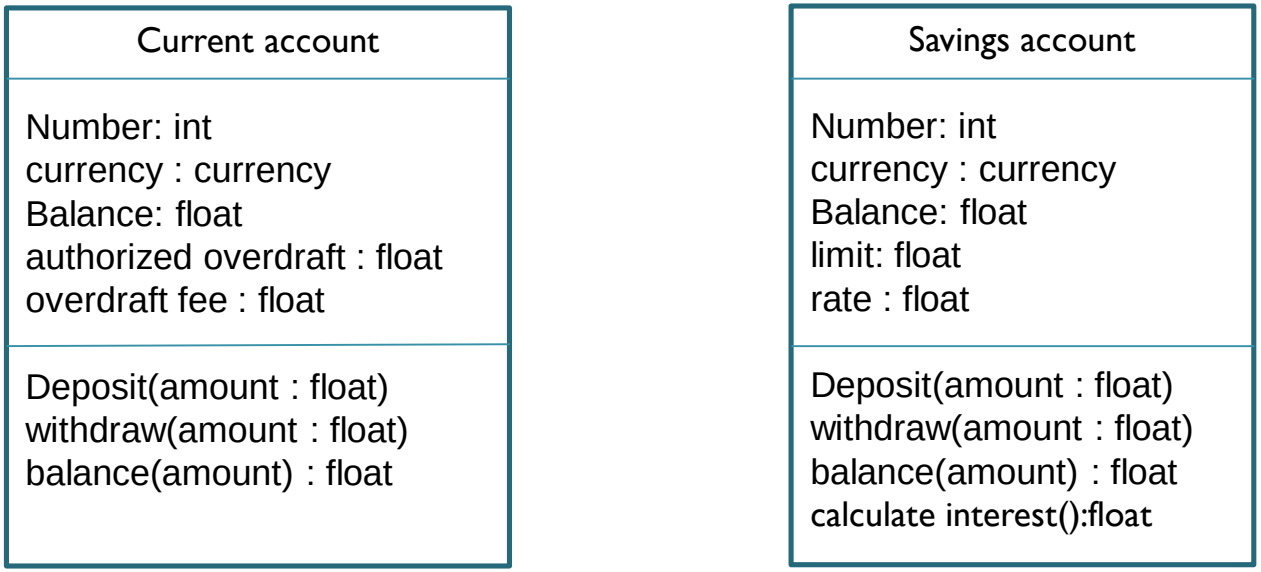
Exemple 2

- This diagram expresses that:
a person was born in a country, and this association cannot be modified;
a person has visited a certain number of countries, in a given order, and the number of visited countries can only increase;
a person would still like to visit a whole list of countries, and this list is ordered (probably by order of preference).



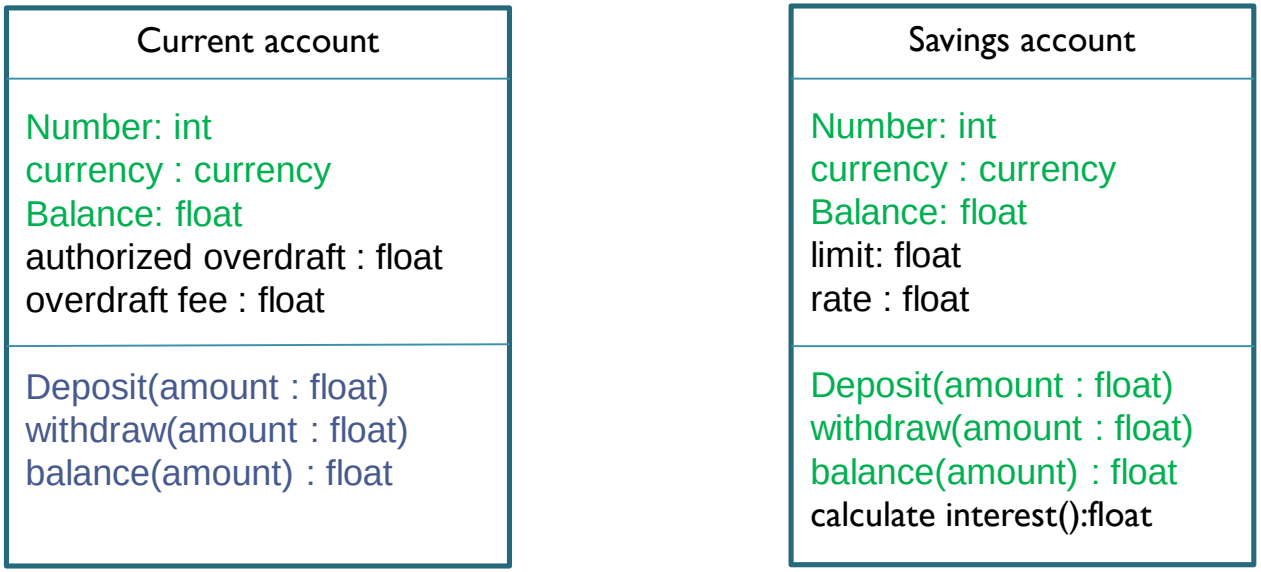
Class hierarchy

Principle: Group classes that share attributes and operations, and organize them in a hierarchy.



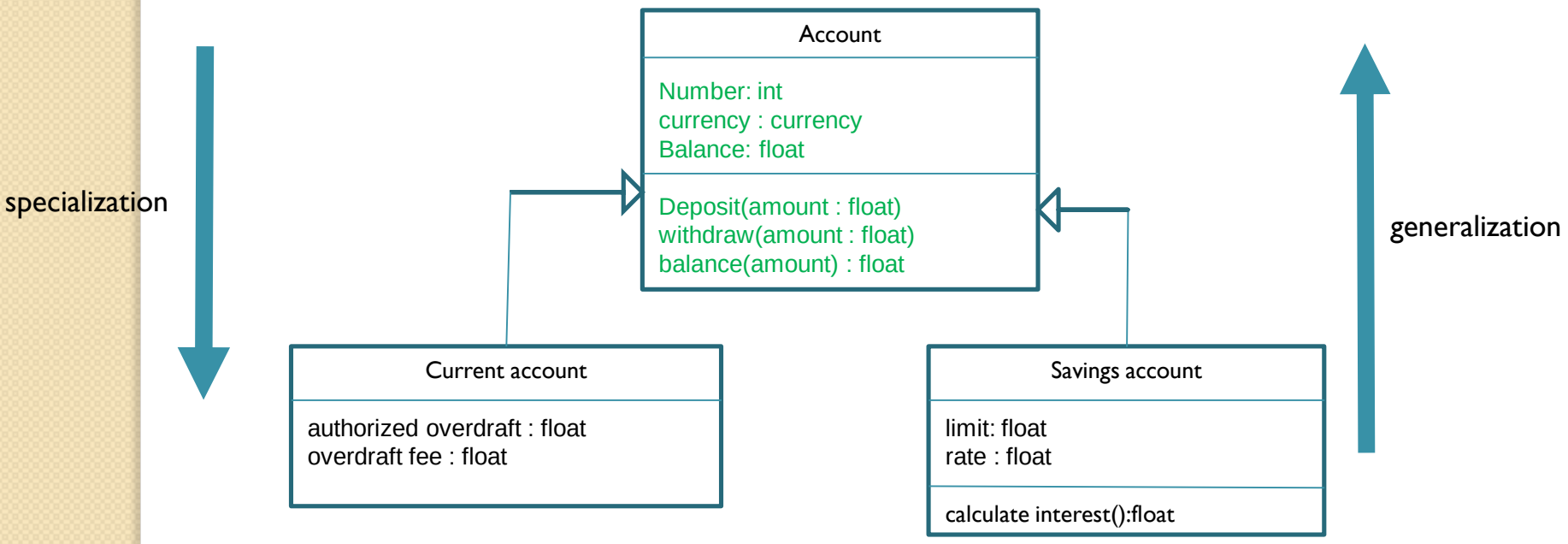
Class hierarchy

Principle: Group classes that share attributes and operations, and organize them in a hierarchy.

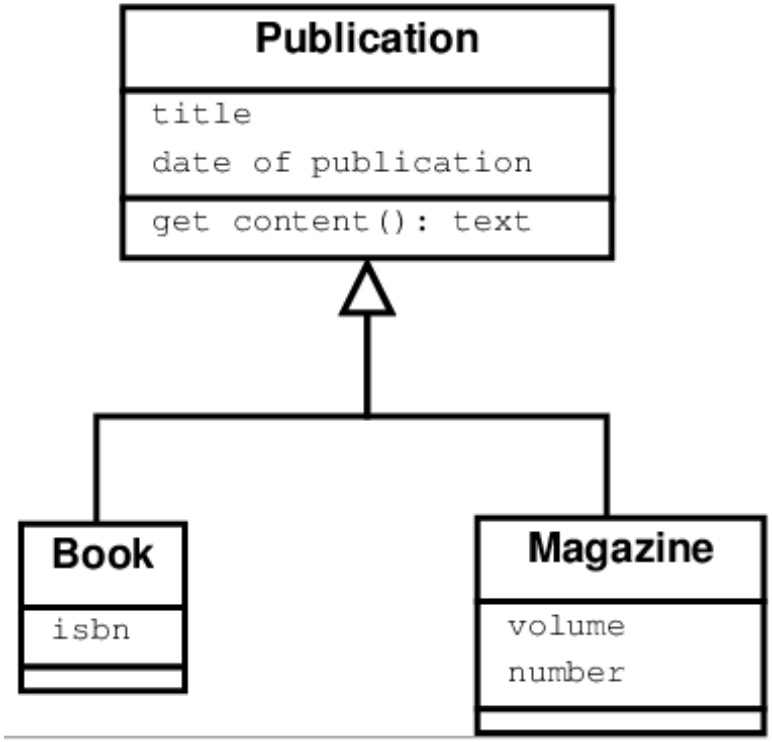


Generalization

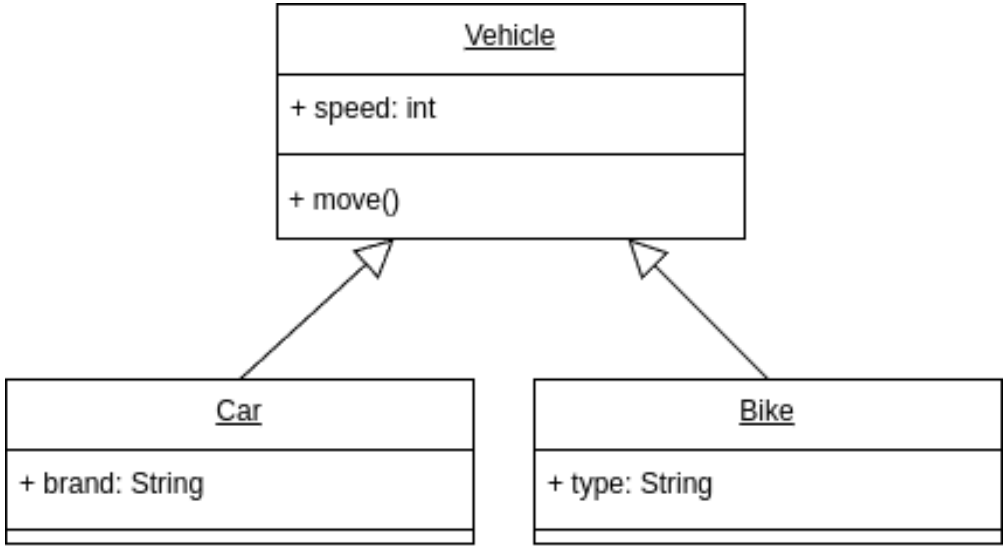
- **Specialization:** refinement of a class into a subclass
  Generalization: abstraction of a set of classes into a superclass



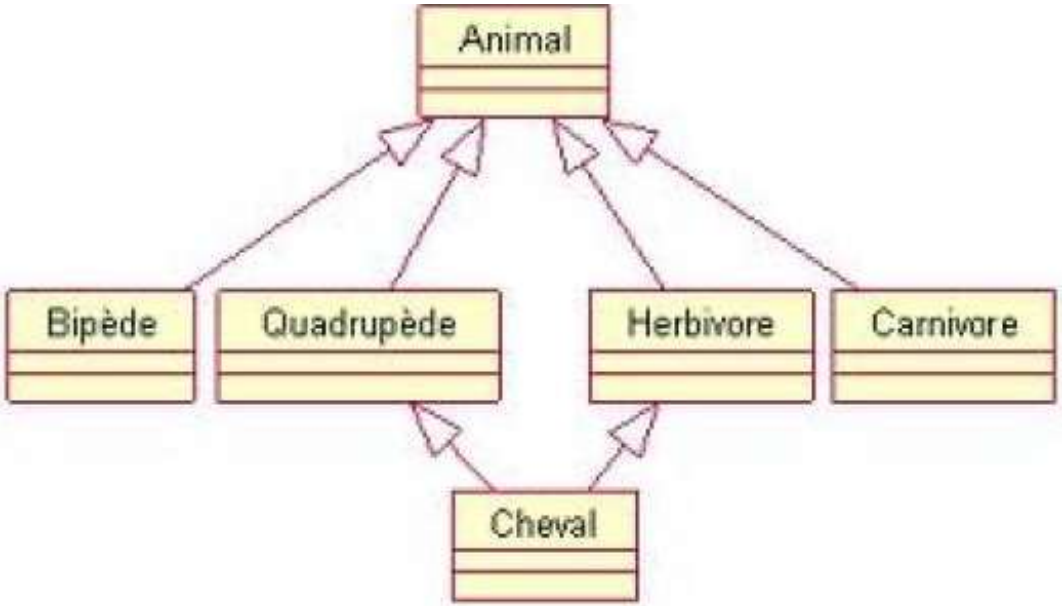
Exemple 1


Publication
title
date of publication
get content(): text
Book
isbn
Magazine
volume
number

Exemple 2



Example of Multiple Generalization

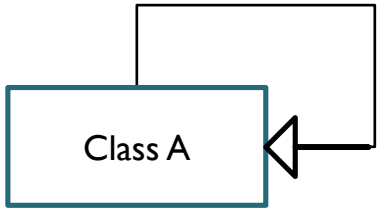


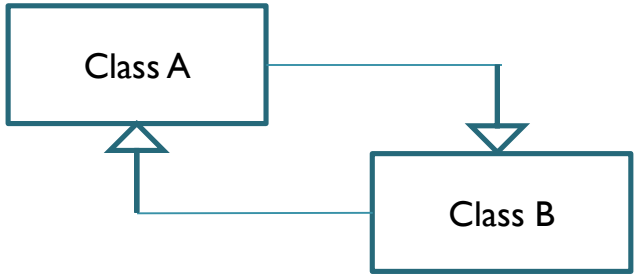
The main properties of inheritance

- The child class inherits all the characteristics of its parent classes, but it cannot access their private attributes;
- a child class can redefine (same signature) one or more methods of the parent class. Unless otherwise specified, an object uses the most specialized operations in the class hierarchy;
- all associations of the parent class apply to the derived classes;
- an instance of a class can be used anywhere an instance of its parent class is expected.
- a class can have several parents; this is called multiple inheritance. C++ is one of the object-oriented languages that allows its effective implementation, while Java does not.

Unauthorized generalizations

- Non-reflexive, non-symmetric relationship!

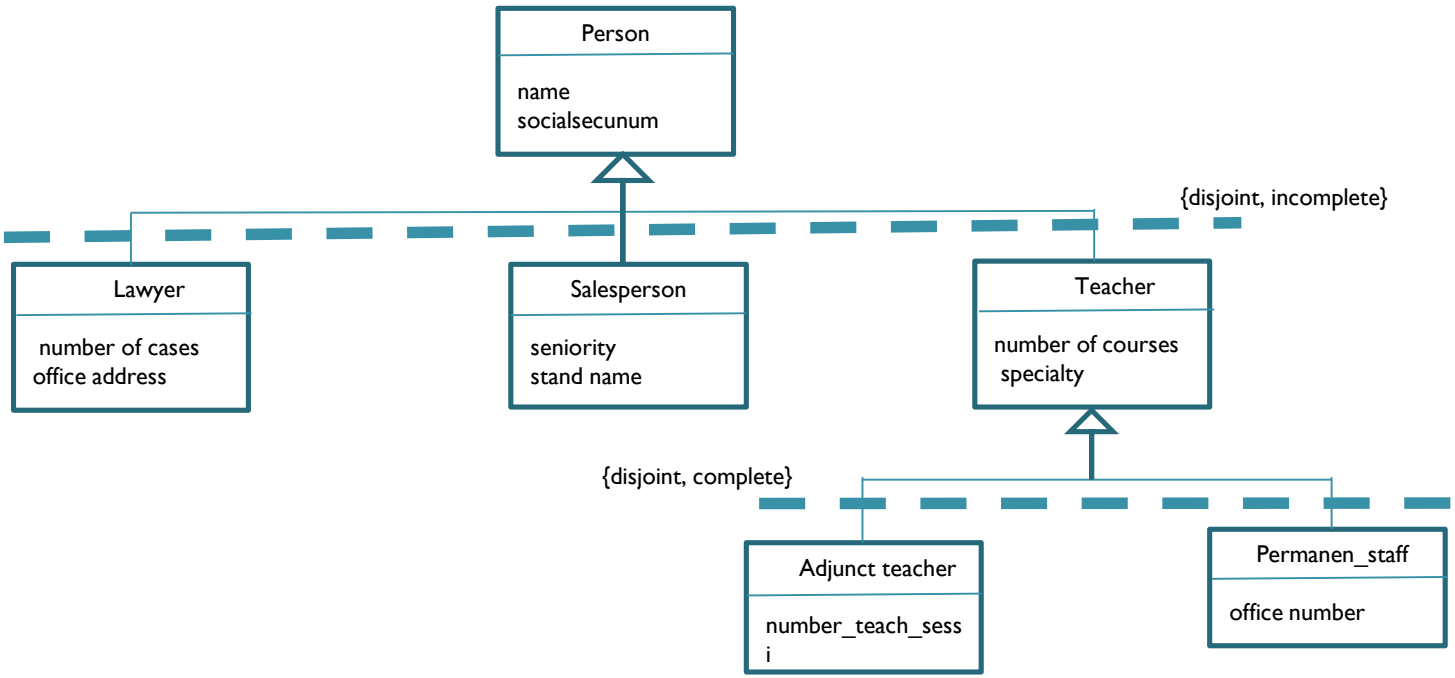




Constraints on inheritance

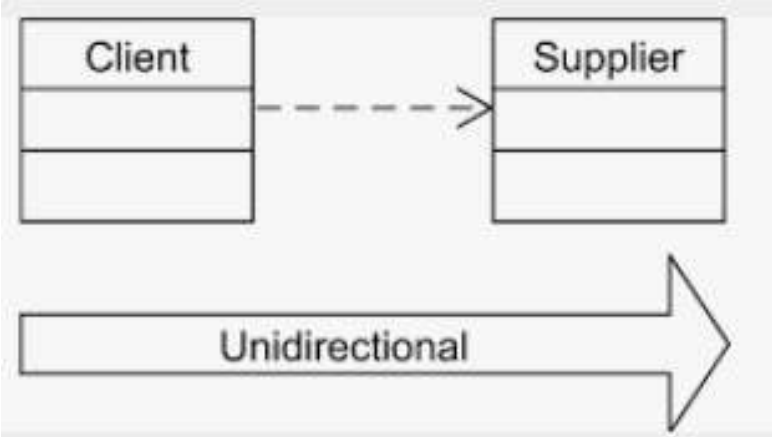
- There are four constraints on the inheritance relationship between a superclass and its subclasses:
- **{incomplete}:** the set of subclasses is incomplete and does not cover the superclass, meaning that the set of instances of the subclasses is a subset of the set of instances of the superclass.
- **{complete}:** the set of subclasses is complete and covers the superclass.
- **{disjoint}:** the subclasses have no instances in common.
- **{overlapping}:** the subclasses may have one or more instances in common.

Exemple


Person
name
socialsecunum
{disjoint, incomplete}
Lawyer
number of cases
office address
Salesperson
seniority
stand name
Teacher
number of courses
specialty
{disjoint, complete}
Adjunct teacher
number_teach_sess
i
Permanen_staff
office number

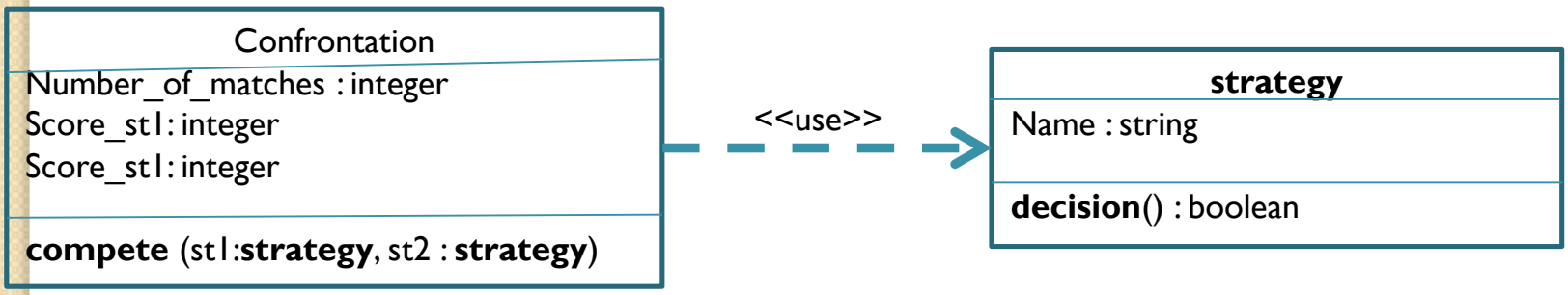
dependency

- A **dependency** is a unidirectional relationship expressing a semantic dependence between model elements.
It indicates that a modification of the target(supplier) may imply a modification of the source (client).
The dependency is often **stereotyped** to better clarify the semantic link between the model elements.
A dependency is often used when one class uses another as an argument in the signature of an operation.
It is represented by a **dashed, directed line**.



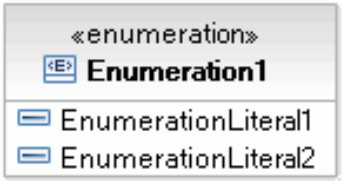
Exemple 1

The **Confrontation** class uses the **Strategy** class because the **Confrontation** class has a method called confronter, whose two parameters are of type **Strategy**.
If the **Strategy** class — particularly its interface — changes, then modifications will also need to be made to the **Confrontation** class.

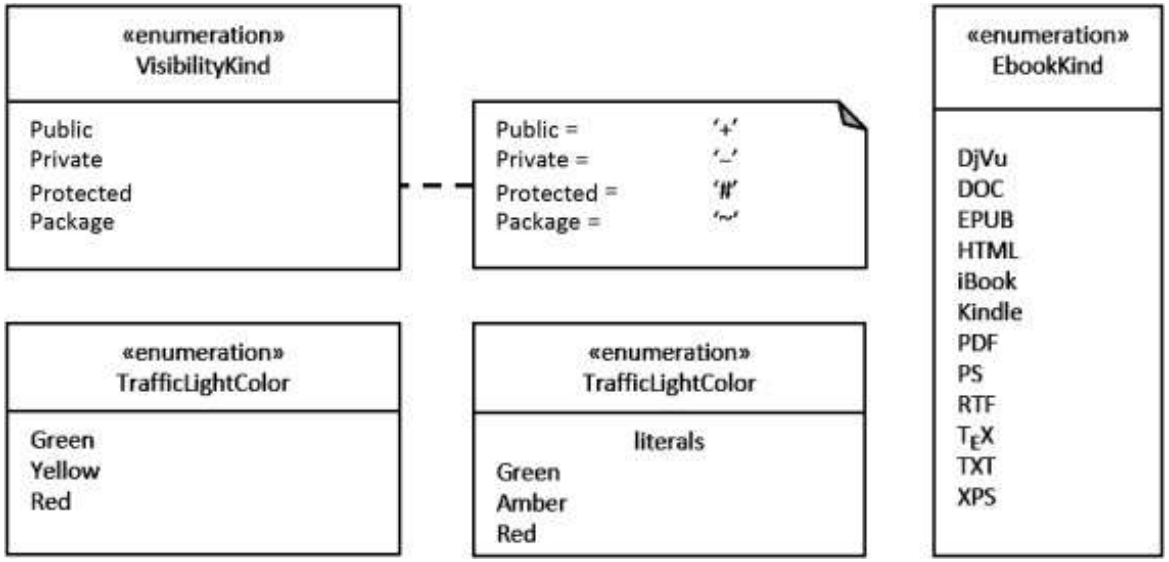


enumeration

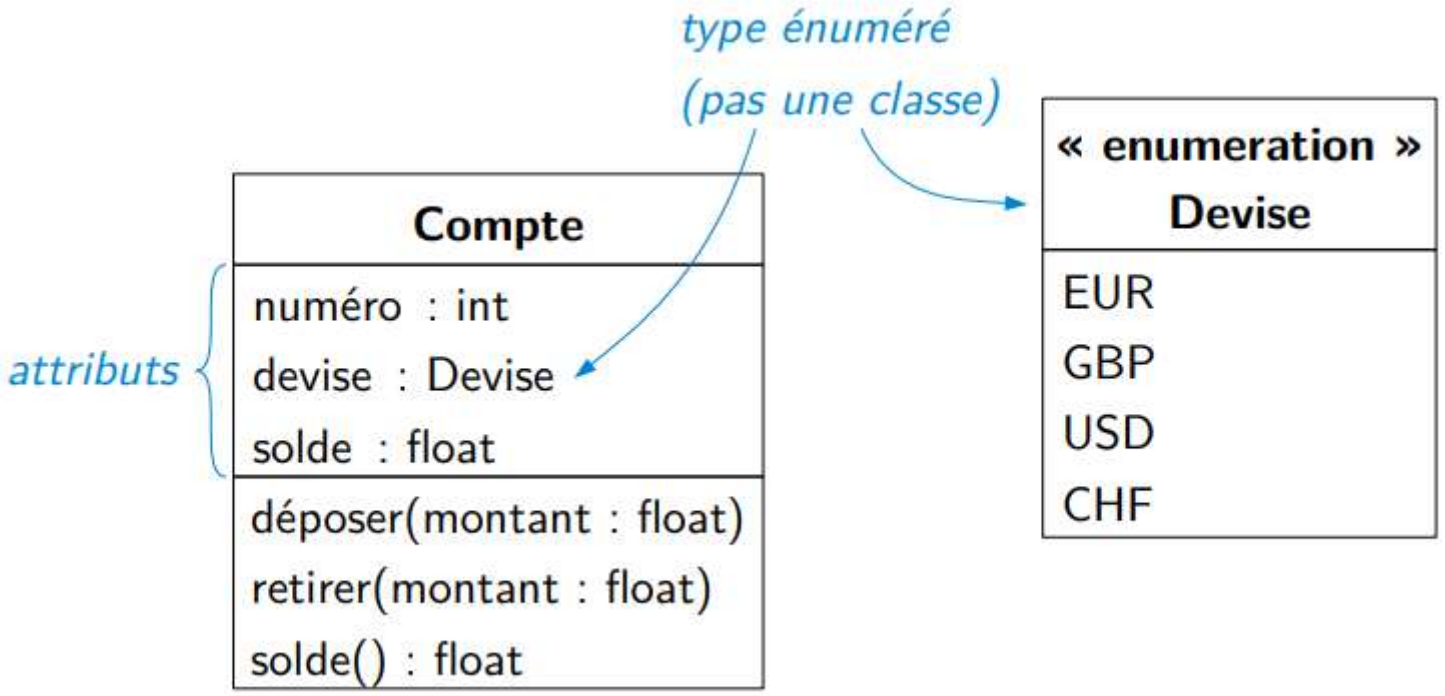
- Enumerations are model elements in class diagrams that represent user-defined data types.
- Enumerations contain sets of named identifiers that represent the values of the enumeration. These values are called enumeration literals.

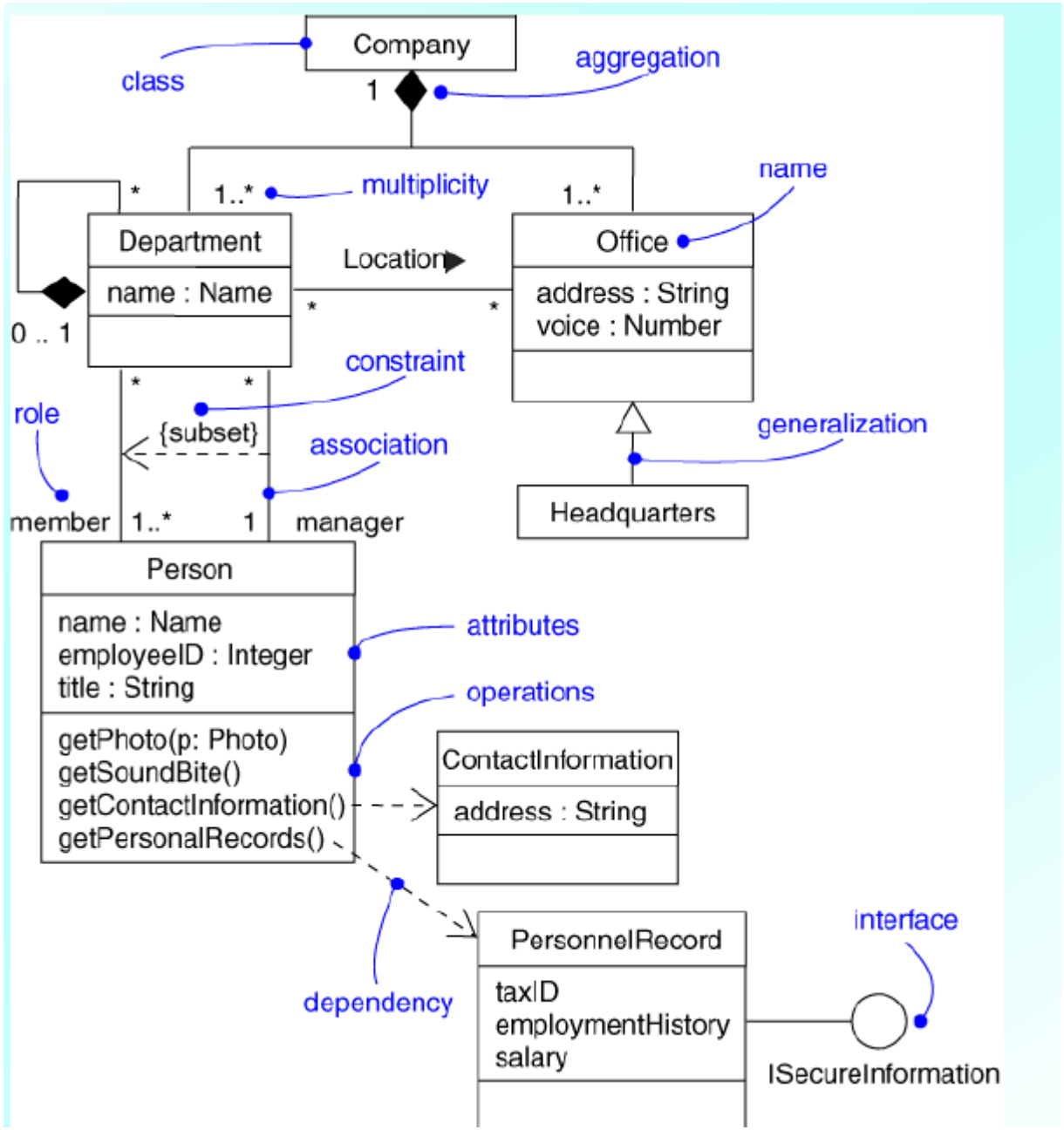


Exemple

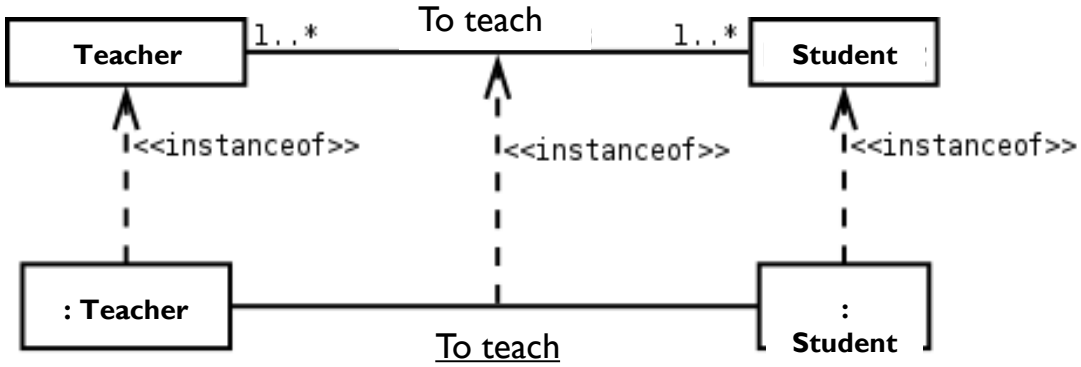

«enumeration»
VisibilityKind
Public
Private
Protected
Package
Public = '+'
Private = '-'
Protected = '#'
Package = '~'
«enumeration»
EbookKind
DjVu
DOC
EPUB
HTML
iBook
Kindle
PDF
PS
RTF
TEX
TXT
XPS
«enumeration»
TrafficLightColor
Green
Yellow
Red
«enumeration»
TrafficLightColor
literals
Green
Amber
Red

Exemple 2

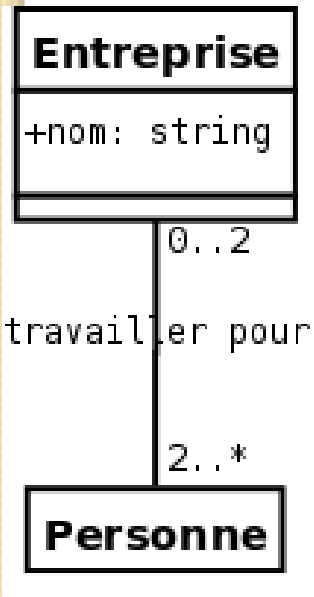

type énuméré
(pas une classe)
« enumeration »
Devise
EUR
GBP
USD
CHF
Compte
attributs
numéro : int
devise : Devise
solde : float
déposer(montant : float)
retirer(montant : float)
solde() : float

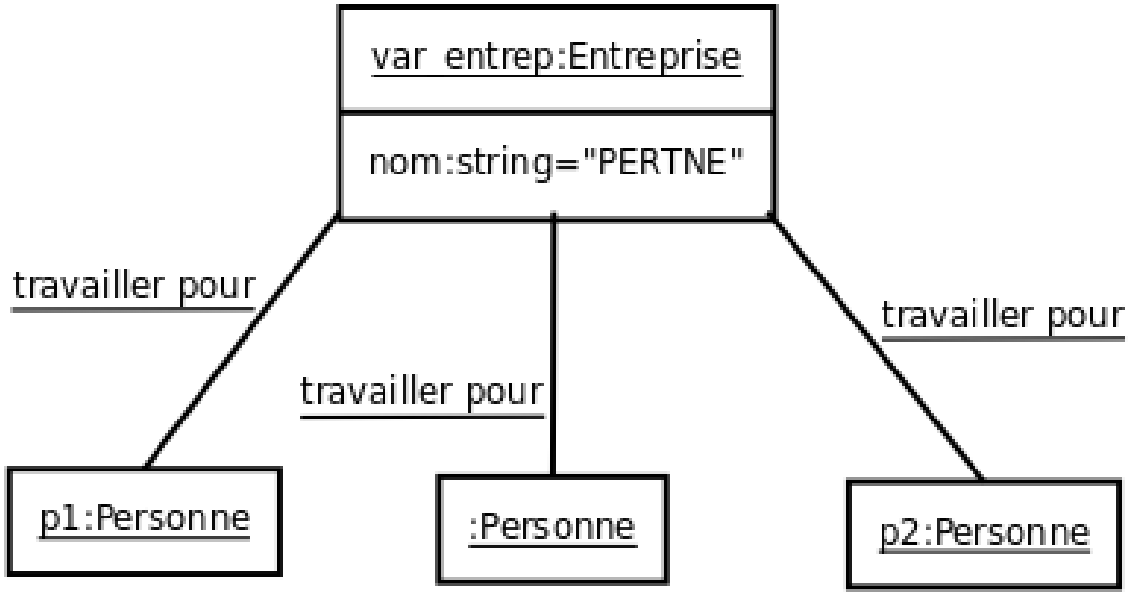

Company
class
aggregation
1
1..*
multiplicity
1..*
name
*
Department
name : Name
Location
Office
address : String
voice : Number
0 .. 1
*
*
*
*
constraint
role
{subset}
association
generalization
member
1..*
1
manager
Headquarters
Person
name : Name
employeeID : Integer
title : String
attributes
operations
getPhoto(p: Photo)
getSoundBite()
getContactInformation()
getPersonalRecords()
ContactInformation
address : String
PersonnelRecord
interface
taxID
employmentHistory
salary
dependency
ISecureInformation

Instantiation dependency relationship

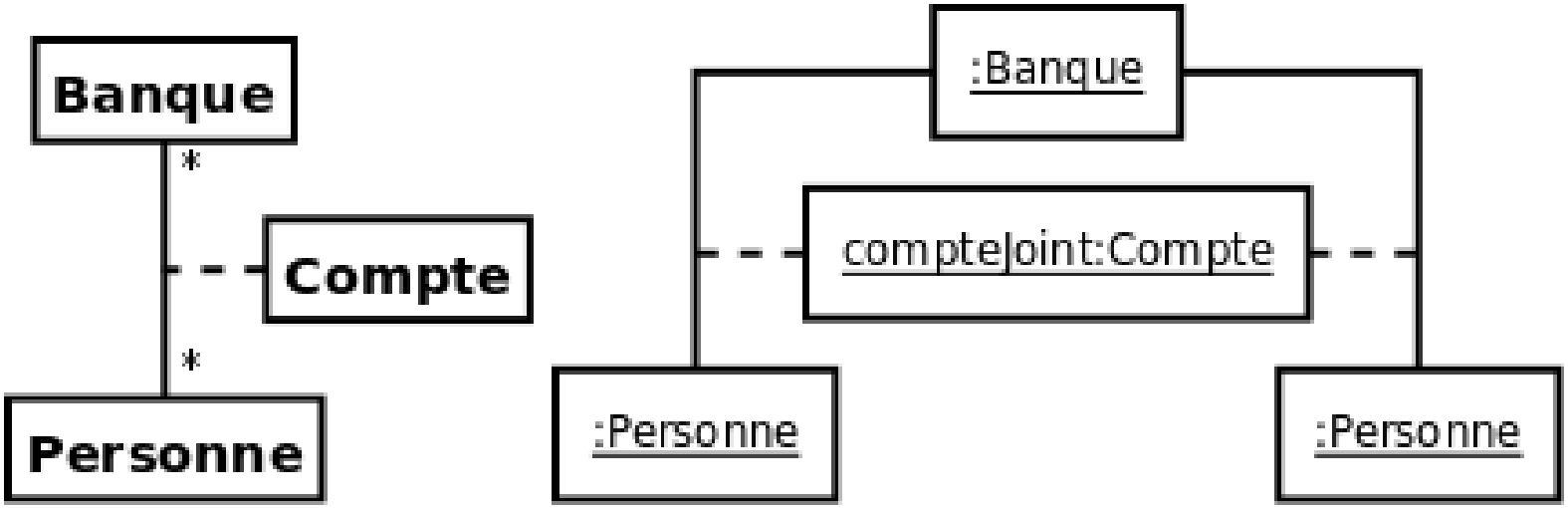


Exemple 1


Entreprise
+nom: string
0..2
travailler pour
2..*
Personne


var_entrep:Entreprise
nom:string="PERTNE"
travailler pour
travailler pour
travailler pour
p1:Personne
:Personne
p2:Personne

Exemple 2


Banque
*
Compte
*
Personne
:Banque
compteJoint:Compte
:Personne
:Personne

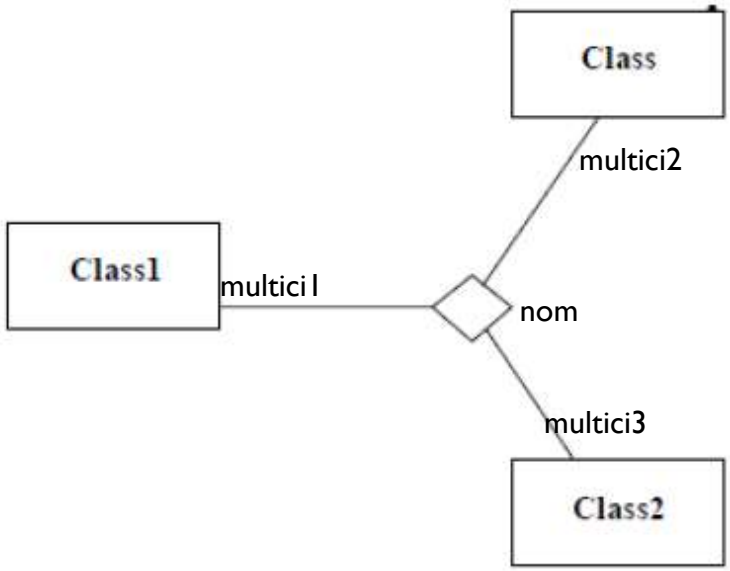
- MERCI DE VOTRE ATTENTION

Constraints on the inheritance relationship

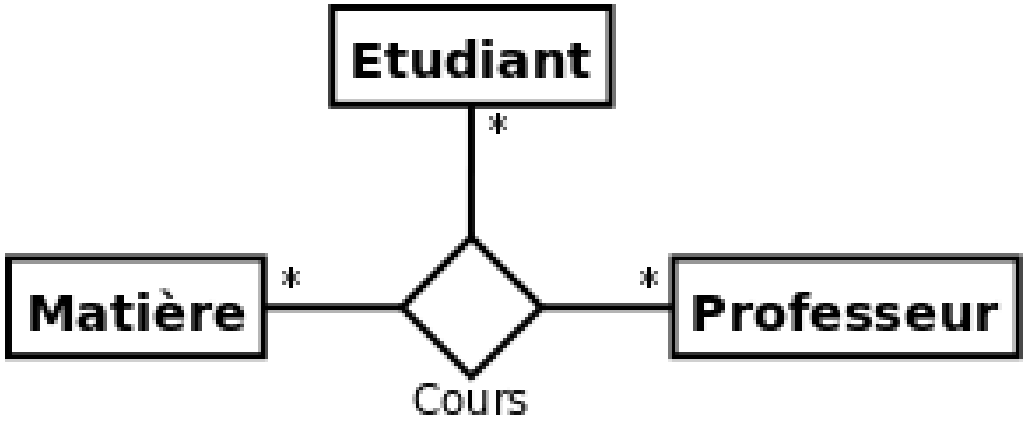
- There are four constraints on the inheritance relationship between a superclass and its subclasses:
- **{incomplete}:** the set of subclasses is incomplete and does not cover the superclass, meaning that the set of instances of the subclasses is a subset of the set of instances of the superclass.
- **{complete}:** the set of subclasses is complete and covers the superclass.
- **{disjoint}:** the subclasses have no instances in common.
- **{overlapping}:** the subclasses may have one or more instances in common.

Association n-aire

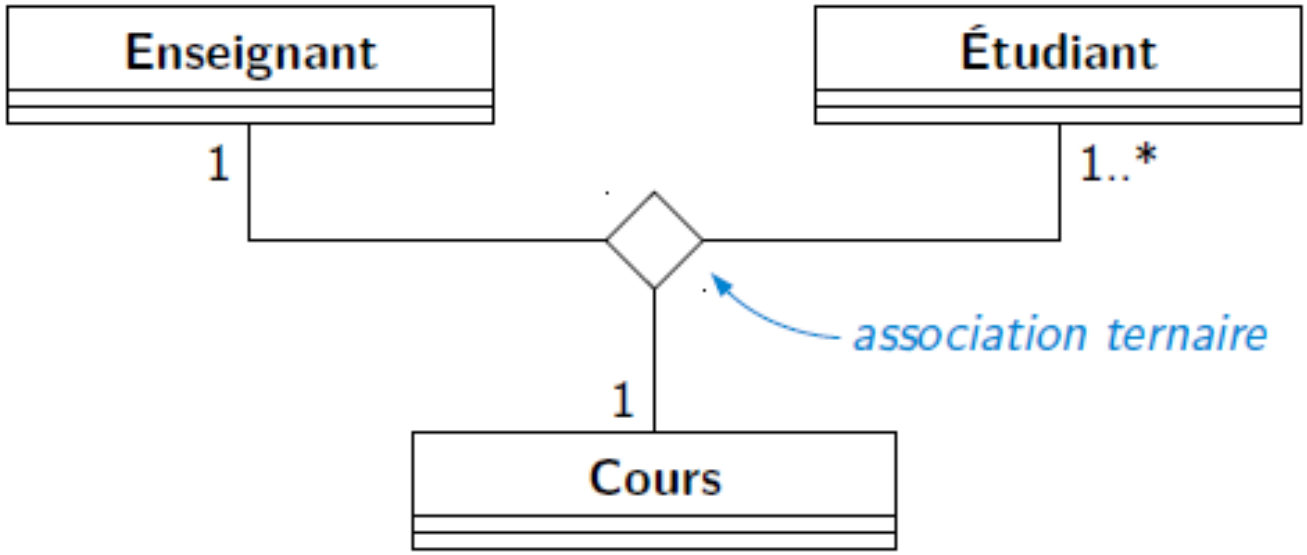
- Une association n-aire lie plus de deux classes.
- On représente une association n-aire par un grand losange avec un chemin partant vers chaque classe participante . Le nom de l'association, le cas échéant, apparaît à proximité du losange.

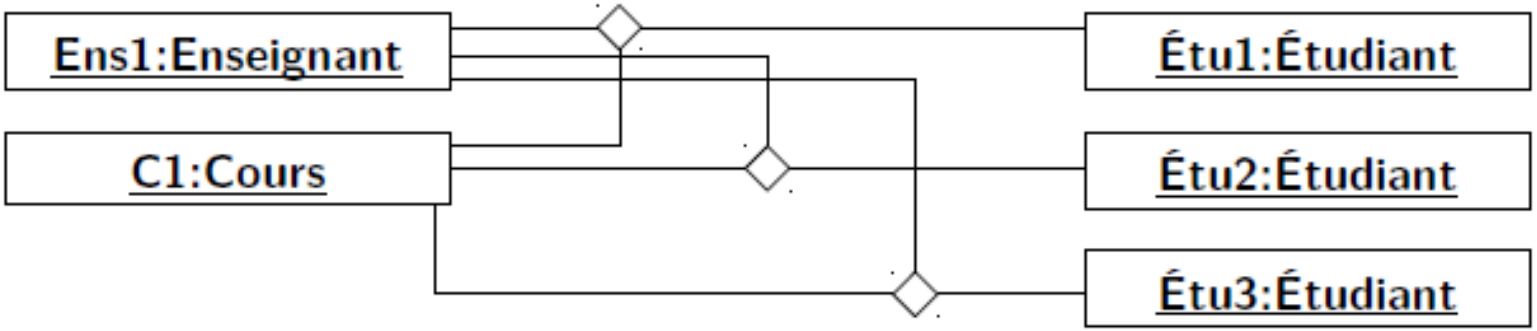


Exemple 1



Exemple 2


Enseignant
Étudiant
1
1..*
association ternaire
1
Cours


Ens1:Enseignant
C1:Cours
Étu1:Étudiant
Étu2:Étudiant
Étu3:Étudiant

Exemple 3

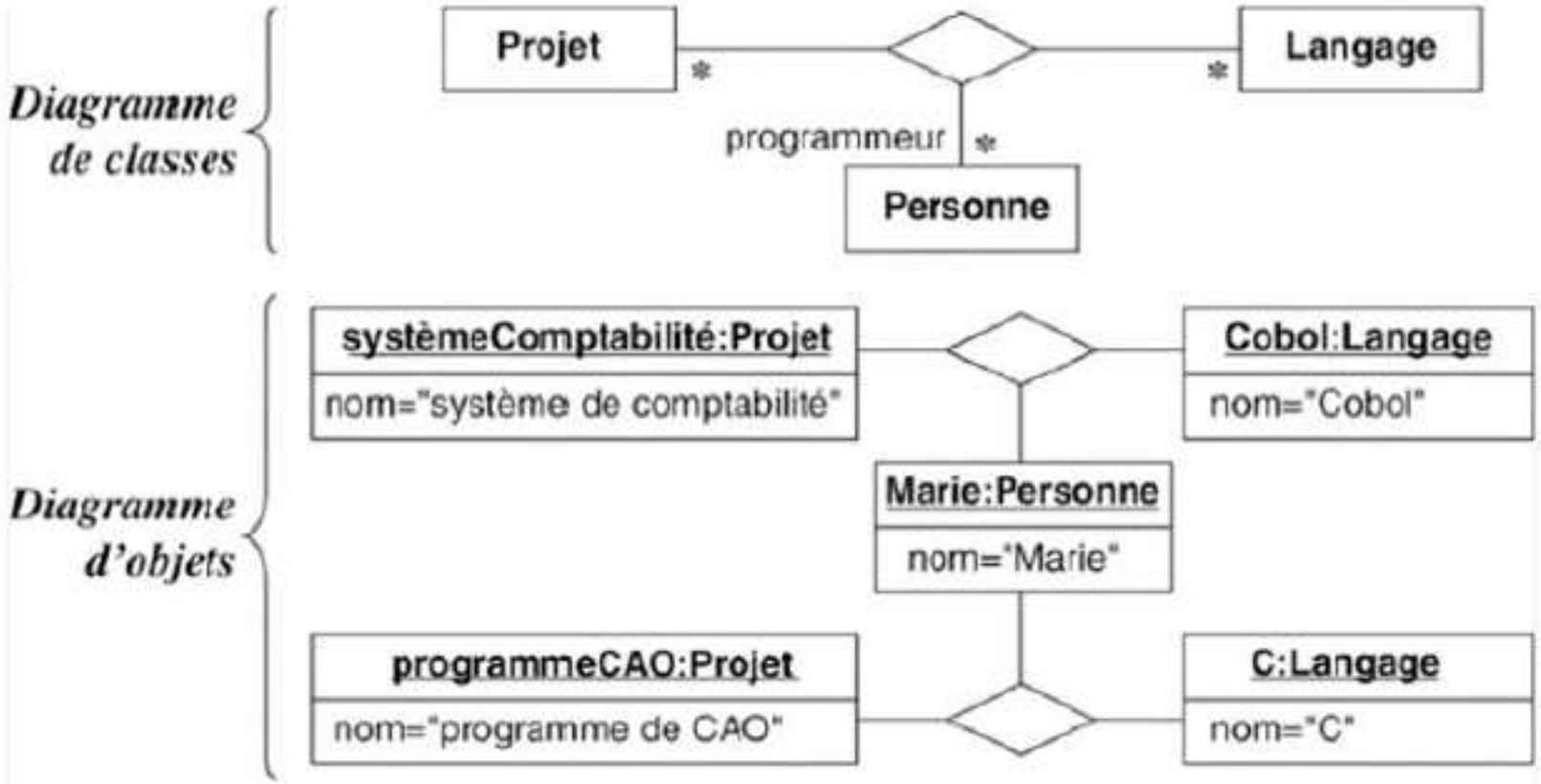
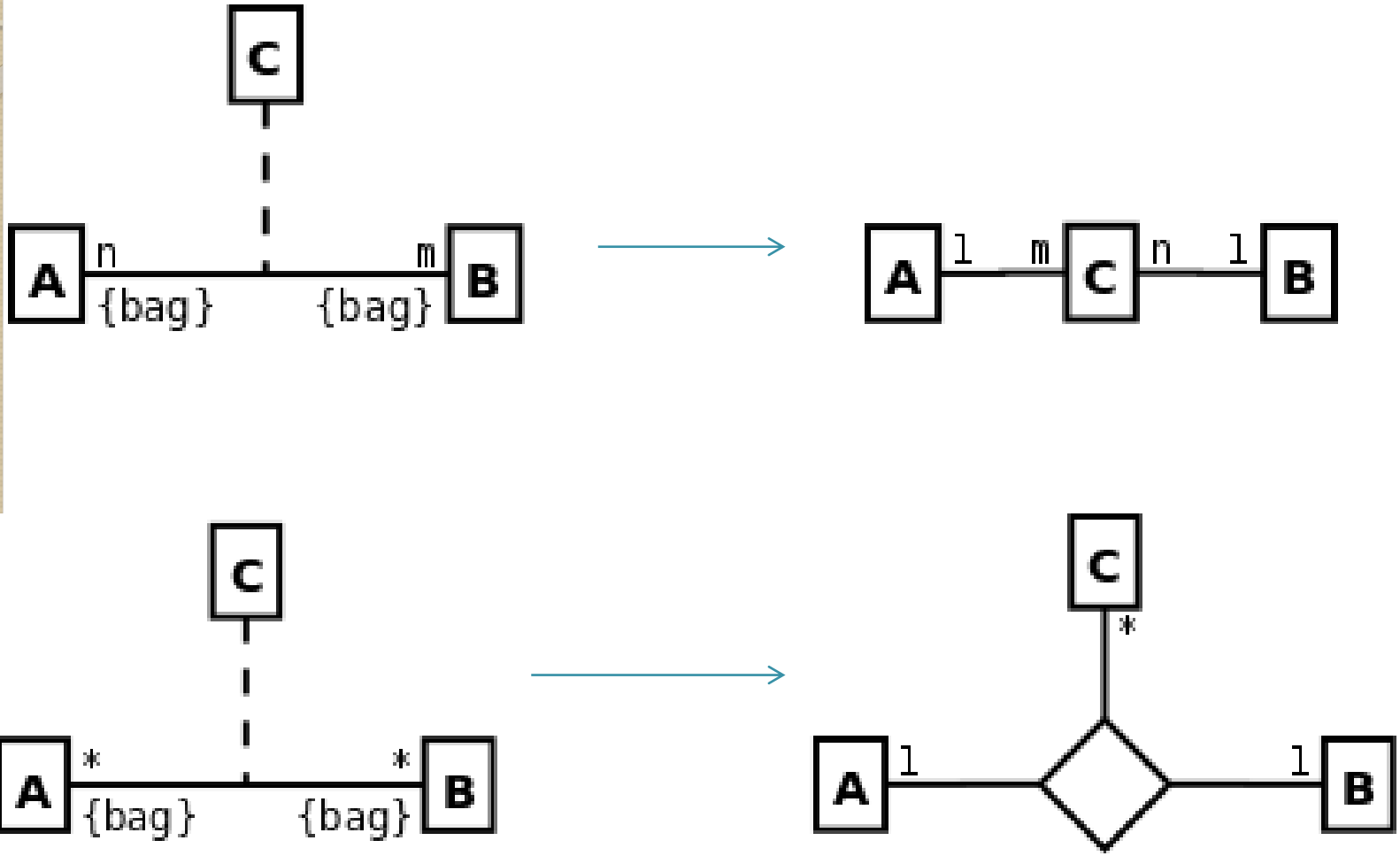
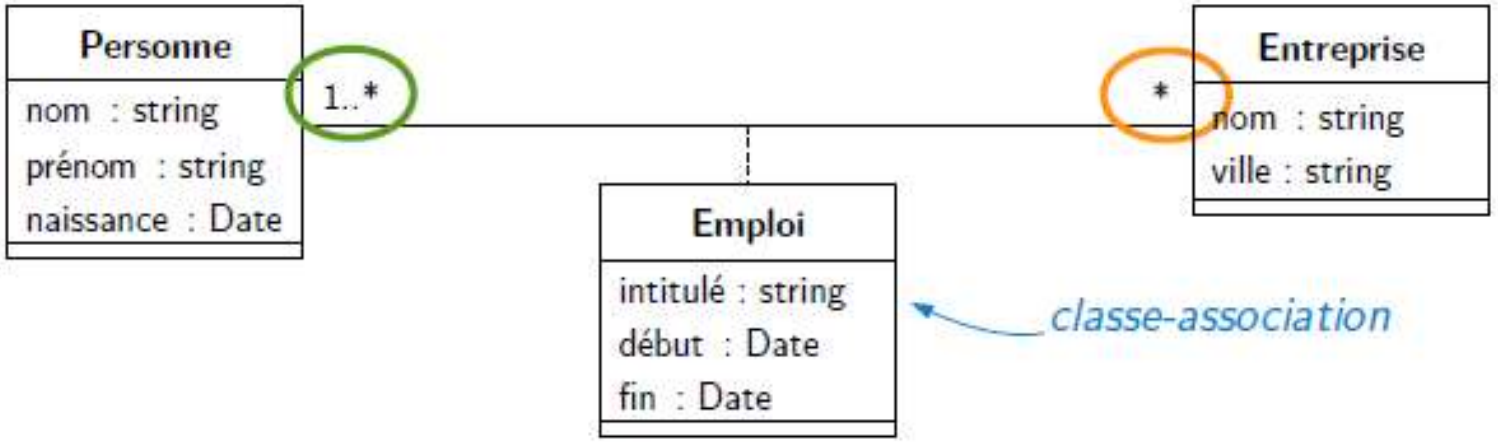


Diagramme de classes
Projet
Langage
programmeur
Personne
Diagramme d'objets
systèmeComptabilité:Projet
nom="système de comptabilité"
Cobol:Langage
nom="Cobol"
Marie:Personne
nom="Marie"
programmeCAO:Projet
nom="programme de CAO"
C:Langage
nom="C"

Équivalences

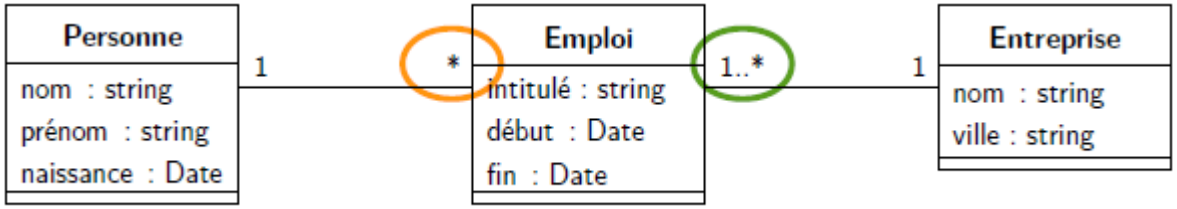


Exemple

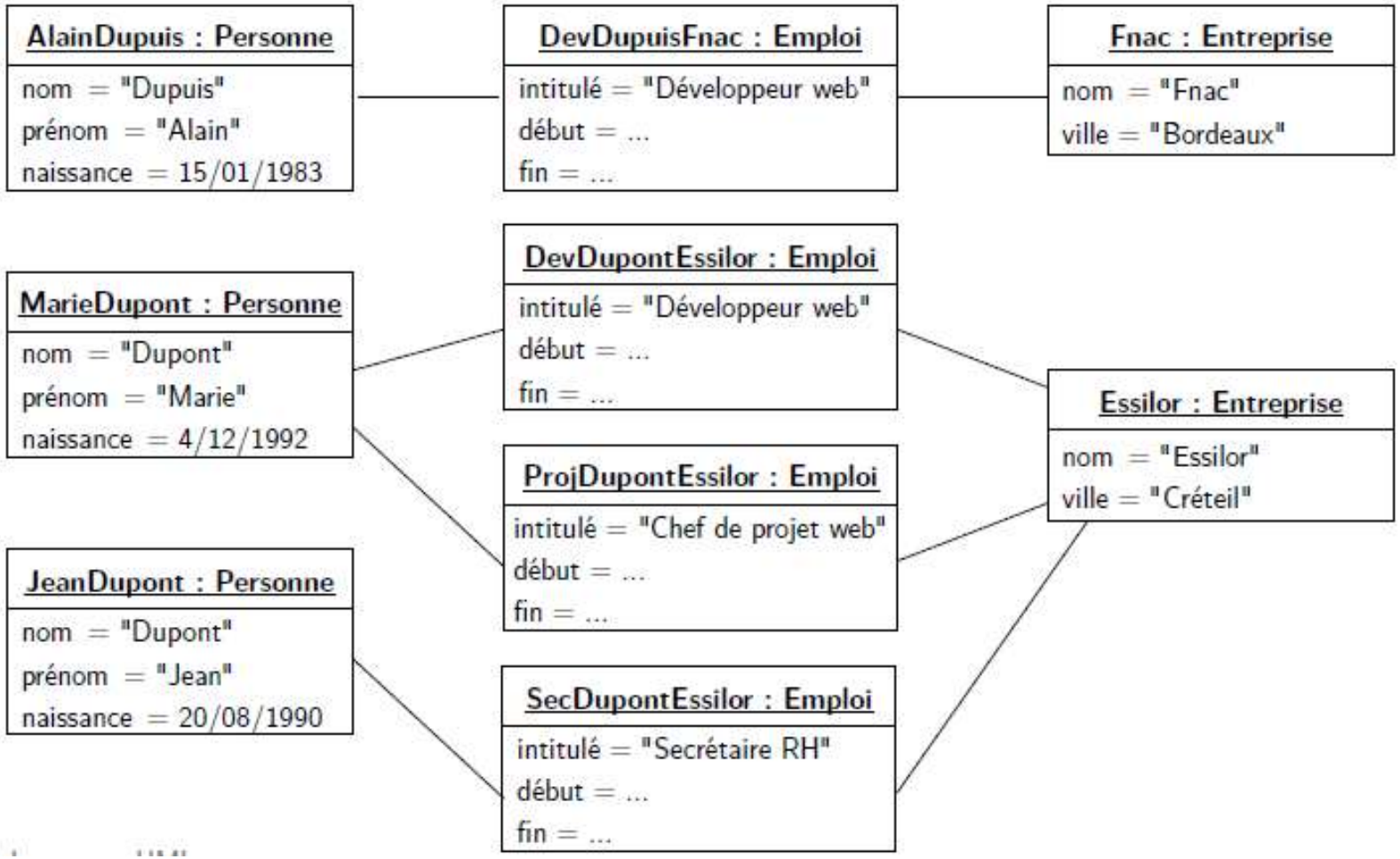


Instance unique de la classe-association pour chaque lien entre objets

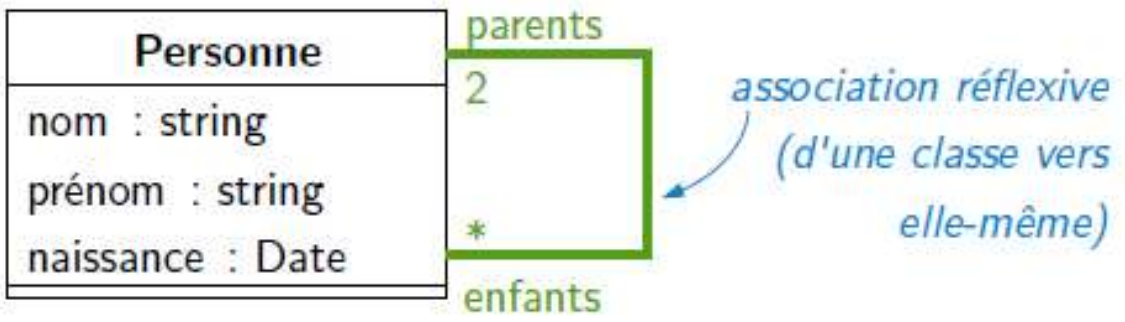
Équivalence en termes de classes et d'associations :

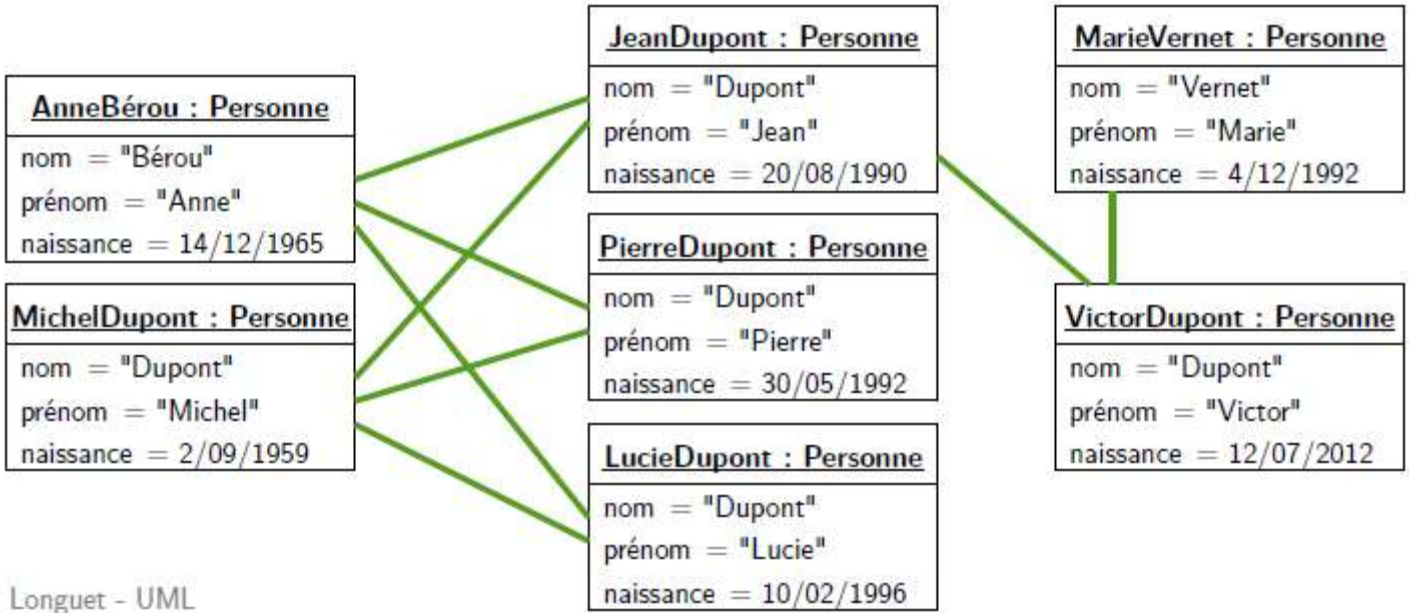


Exemple


AlainDupuis : Personne
nom = "Dupuis"
prénom = "Alain"
naissance = 15/01/1983
DevDupuisFnac : Emploi
intitulé = "Développeur web"
début = ...
fin = ...
Fnac : Entreprise
nom = "Fnac"
ville = "Bordeaux"
MarieDupont : Personne
nom = "Dupont"
prénom = "Marie"
naissance = 4/12/1992
DevDupontEssilor : Emploi
intitulé = "Développeur web"
début = ...
fin = ...
Essilor : Entreprise
nom = "Essilor"
ville = "Créteil"
ProjDupontEssilor : Emploi
intitulé = "Chef de projet web"
début = ...
fin = ...
JeanDupont : Personne
nom = "Dupont"
prénom = "Jean"
naissance = 20/08/1990
SecDupontEssilor : Emploi
intitulé = "Secrétaire RH"
début = ...
fin = ...

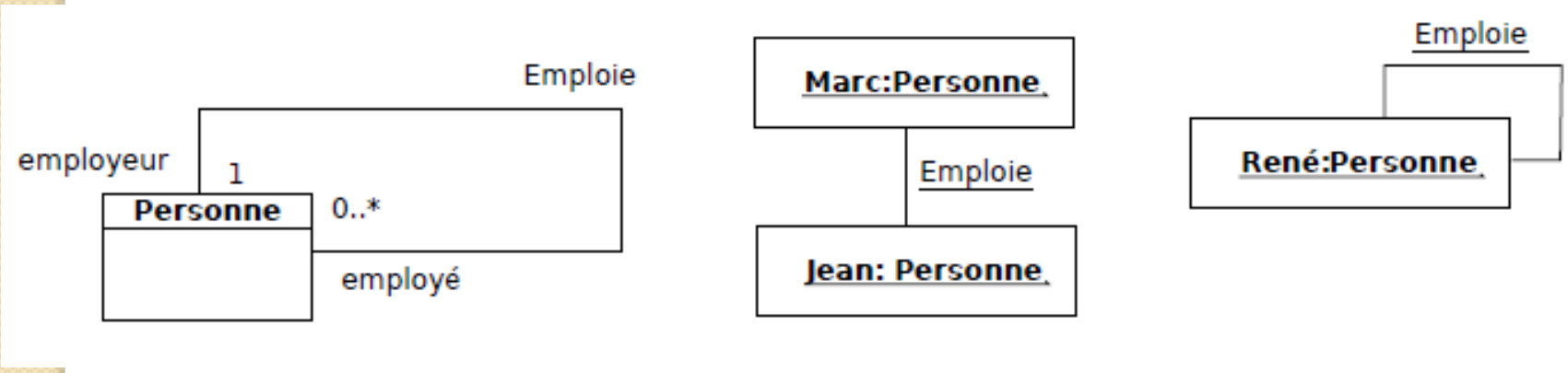
Association réflexive





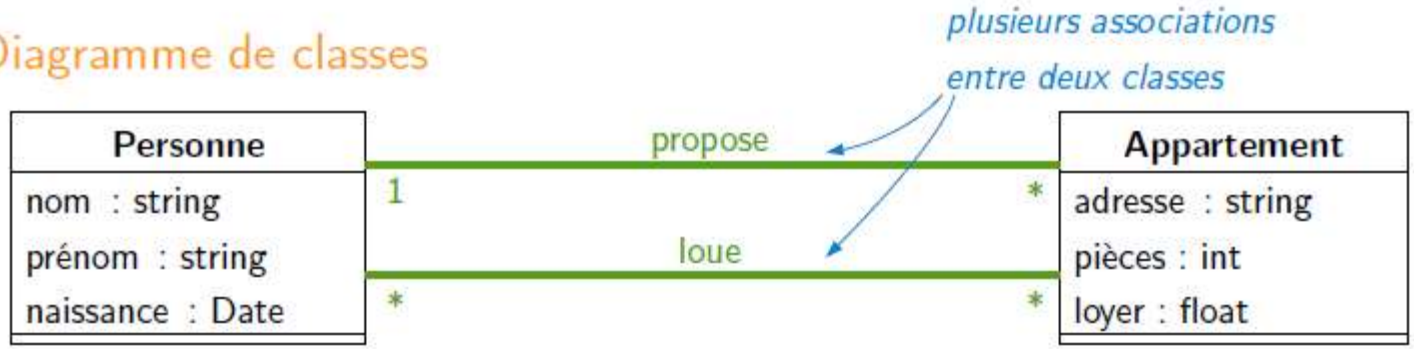
Association réflexive

- Une instance de la classe impliquée peut être reliée a elle-même ou a d'autres instances de cette même classe.

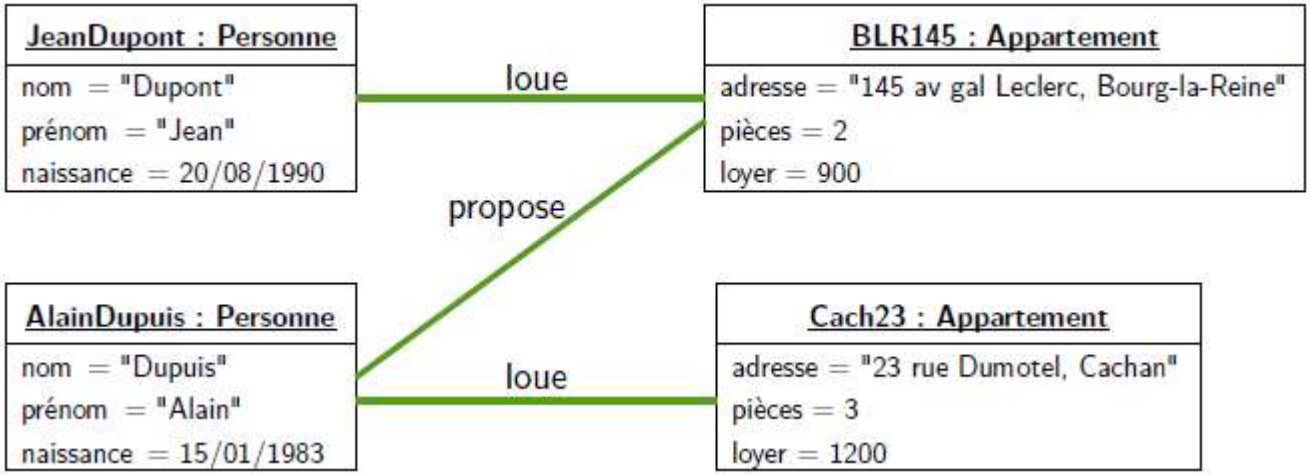


Association multiple

Diagramme de classes

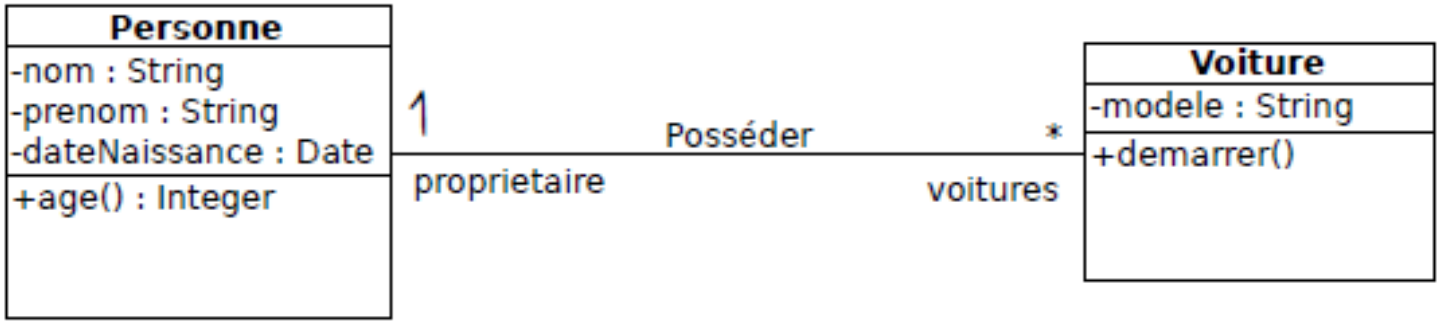


Exemple de diagramme d'objets



Association

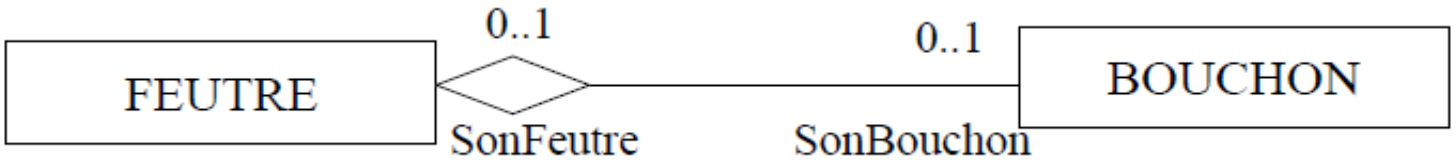
- Une autre façons de modéliser une association



Agrégation

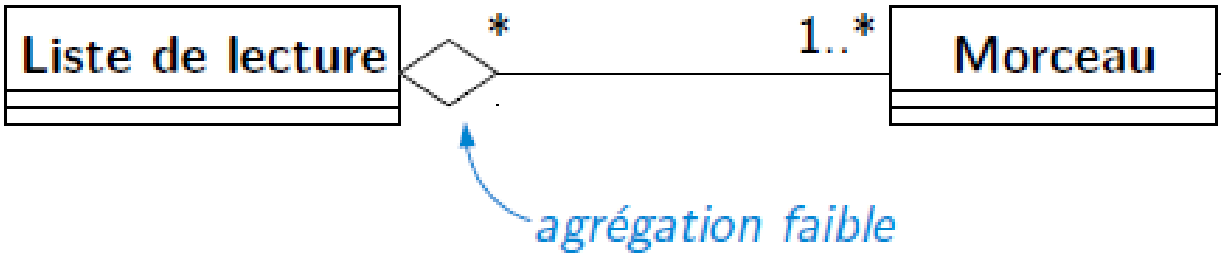
- Une agrégation est un cas particulier d'association non symétrique exprimant une relation de **contenance** d 'un élément dans un ensemble **(tout/partie**).
- Les agrégations n'ont pas besoin d'être nommées : implicitement elles signifient «contient», «est composéde».
- On représente l'agrégation par l'ajout d'un losange vide du côté de l'agrégat (l'ensemble).

Exemple



- Un feutre peut perdre son bouchon et un bouchon peut perdre le corps de son feutre d'origine. Il n'y a pas forcément cohérence entre les cycles de vie des feutres et des bouchons.

Exemples

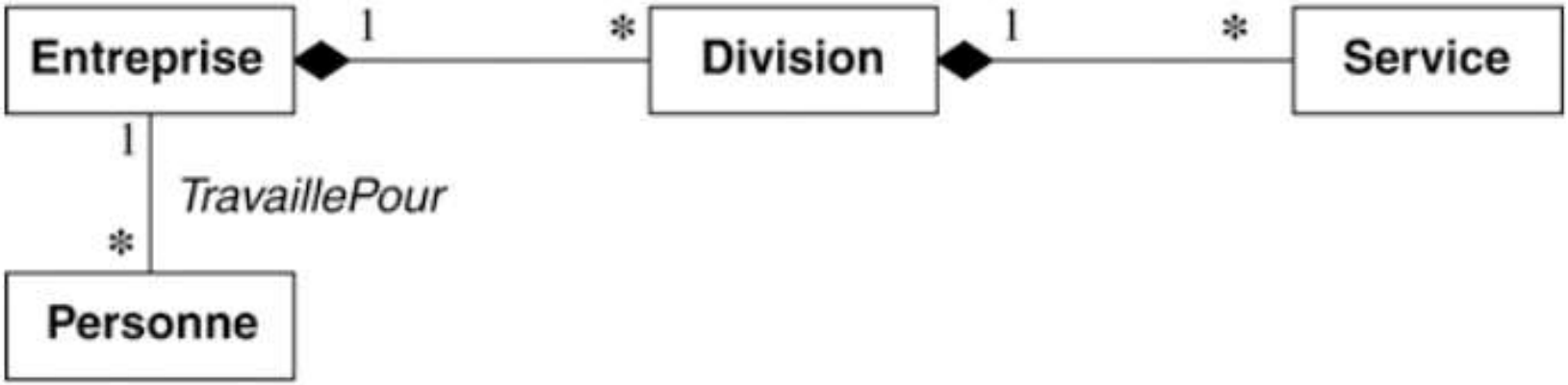


Composition

- Une composition est une agrégation plus forte impliquant que :
  - un élément ne peut appartenir qu'à un seul agrégat composite (agrégation non partagée).
  - la destruction de l'agrégat composite (l'ensemble) entraîne la destruction de tous ses éléments (les parties).

- le composite est responsable du cycle de vie des parties.

Exemples


Entreprise
1
*
Division
1
*
Service
1
*
TravaillePour
Personne

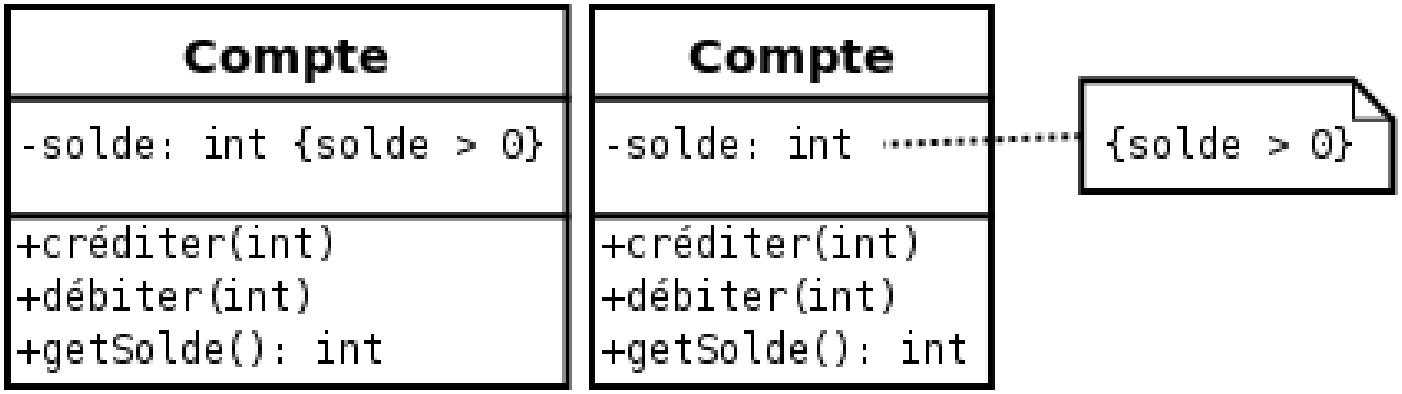

Livre
1
1..n
Chapitre
1
1..n
Paragraphe

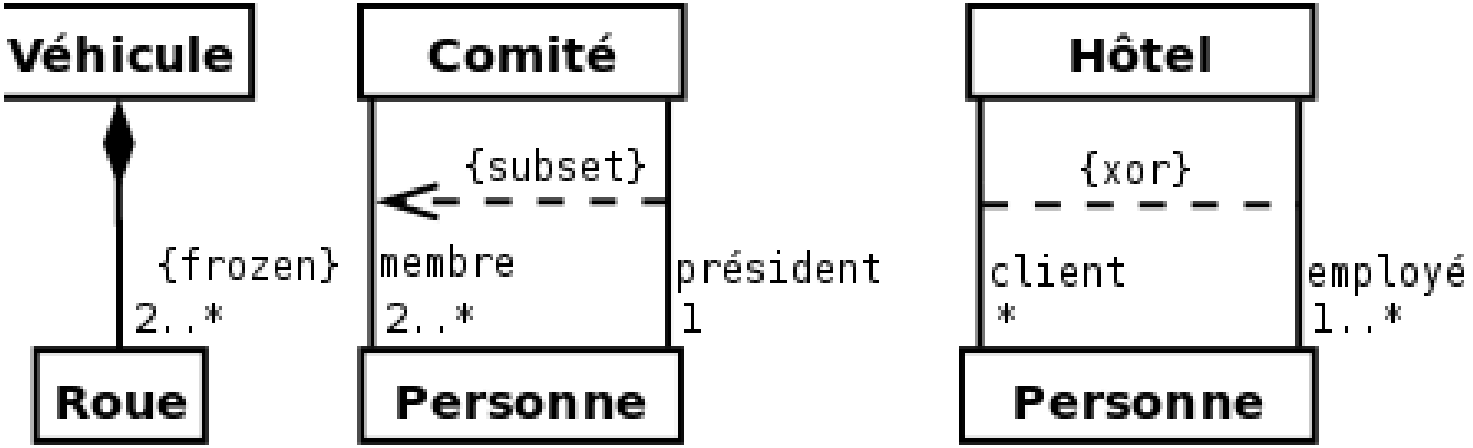
Contrainte

- Propriétés sur extrémités d'associations
  - {variable} : instance modifiable (par défaut)
  - {frozen} : instance non modifiable
  - {addOnly} : instances ajoutables mais non retirables (si mult. > 1)

- Contraintes prédéfinies
  - Sur une extrémité : {ordered}, {unique}, ...
  - Entre 2 associations : {subset}, {xor}, ...

Exemple 1


Compte
-solde: int {solde > 0}
+créditer(int)
+débiter(int)
+getSolde(): int
Compte
-solde: int
+créditer(int)
+débiter(int)
+getSolde(): int
{solde > 0}


Véhicule
{frozen}
2..*
Roue
Comité
{subset}
membre
2..*
président
1
Personne
Hôtel
{xor}
client
*
employé
1..*
Personne

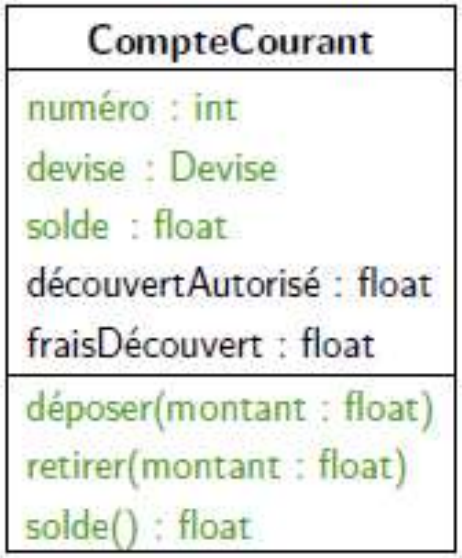
Exemple 2

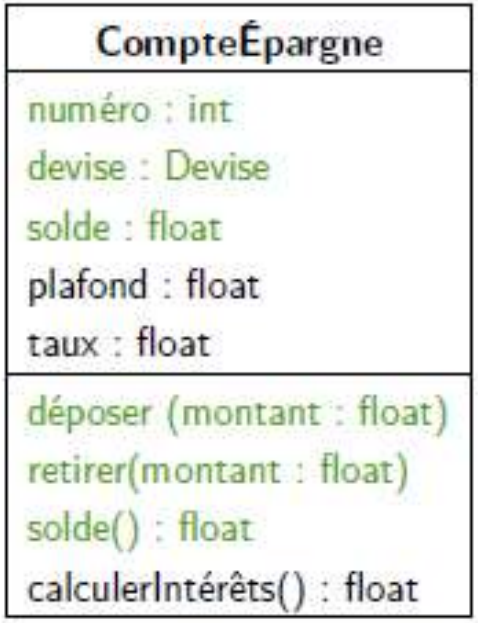
Ce diagramme exprime que :

- une personne est née dans un pays, et que cette association ne peut être modifiée ;
- une personne a visité un certain nombre de pays, dans un ordre donné, et que le nombre de pays visités ne peut que croître ;
- une personne aimerait encore visiter toute une liste de pays, et que cette liste est ordonnée (probablement par ordre de préférence).

Hiérarchie de classes

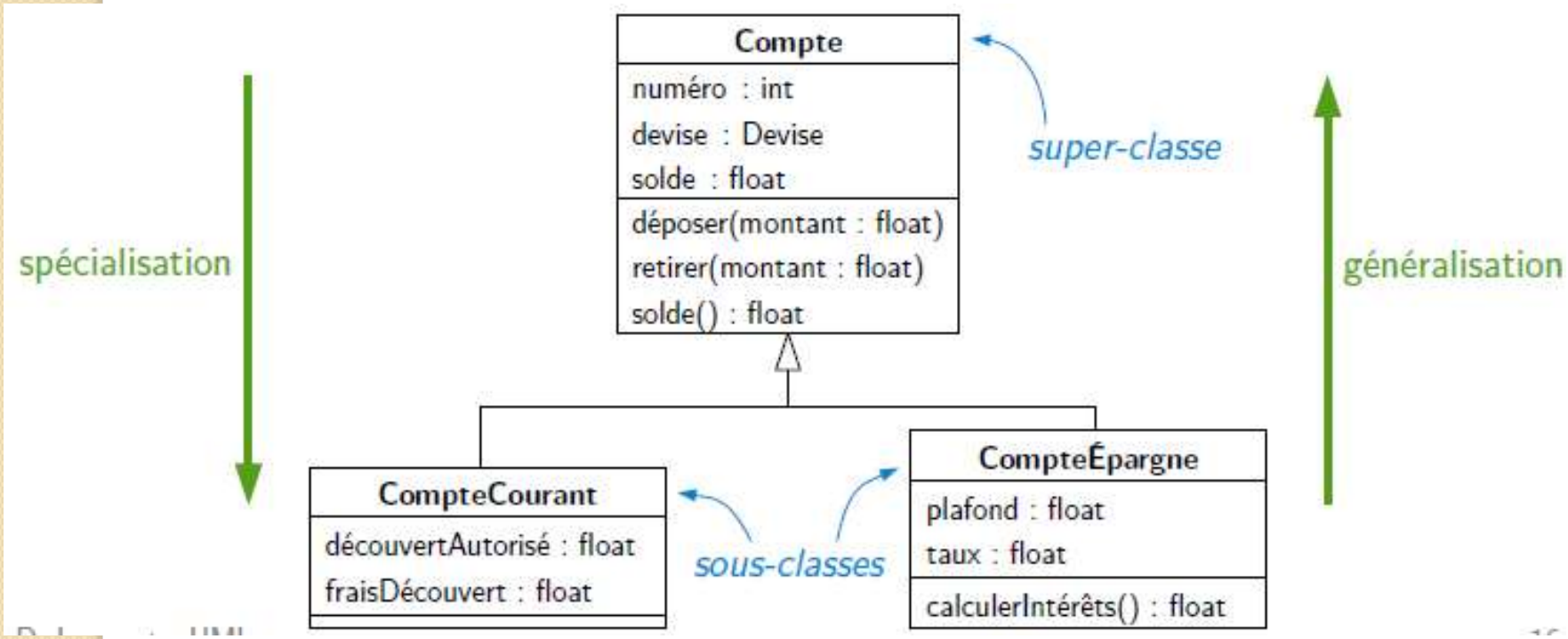
- Principe : Regrouper les classes partageant des attributs et des opérations et les organiser en arborescence



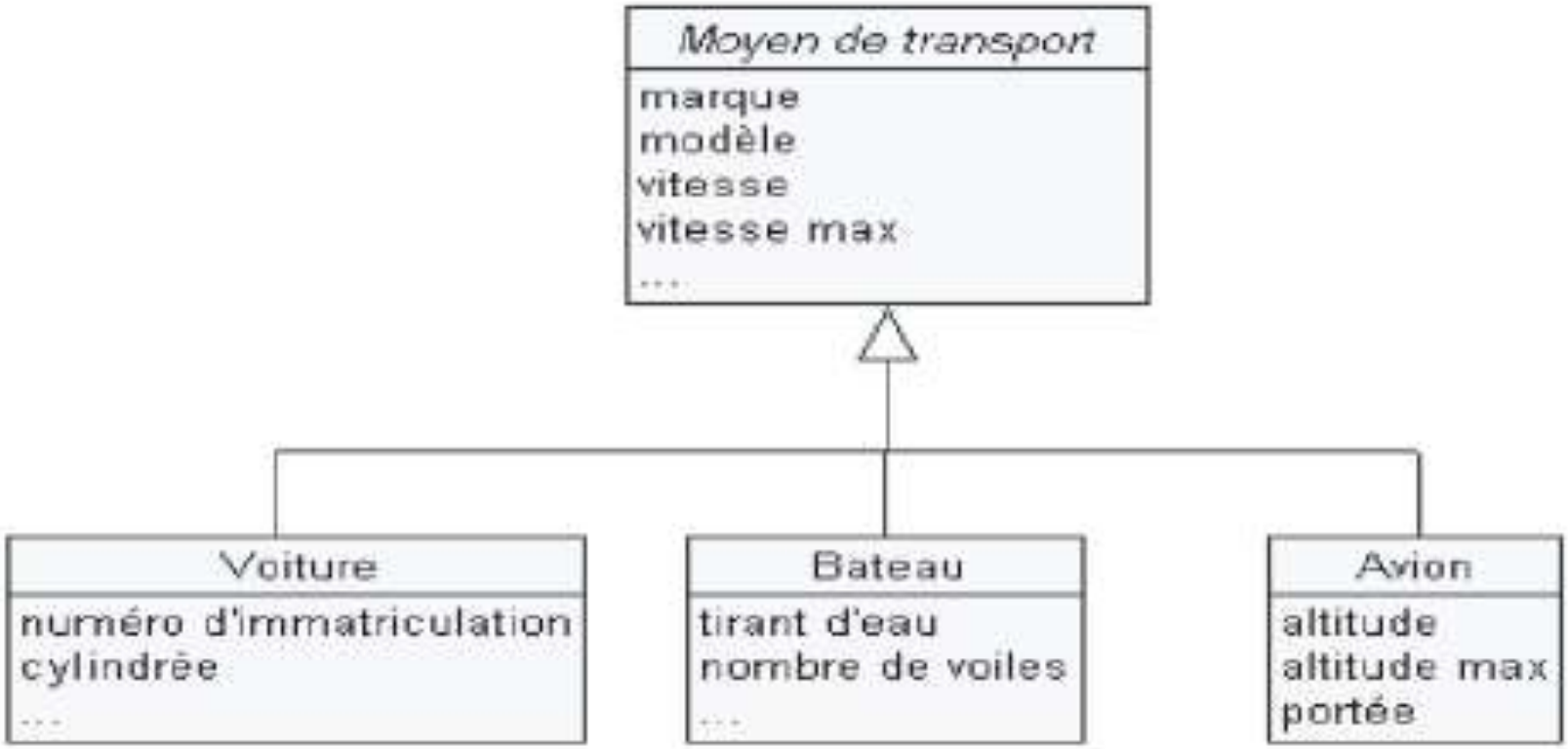


Généralisation

- Spécialisation : raffinement d'une classe en une sous-classe
- Généralisation : abstraction d'un ensemble de classes en super-classe

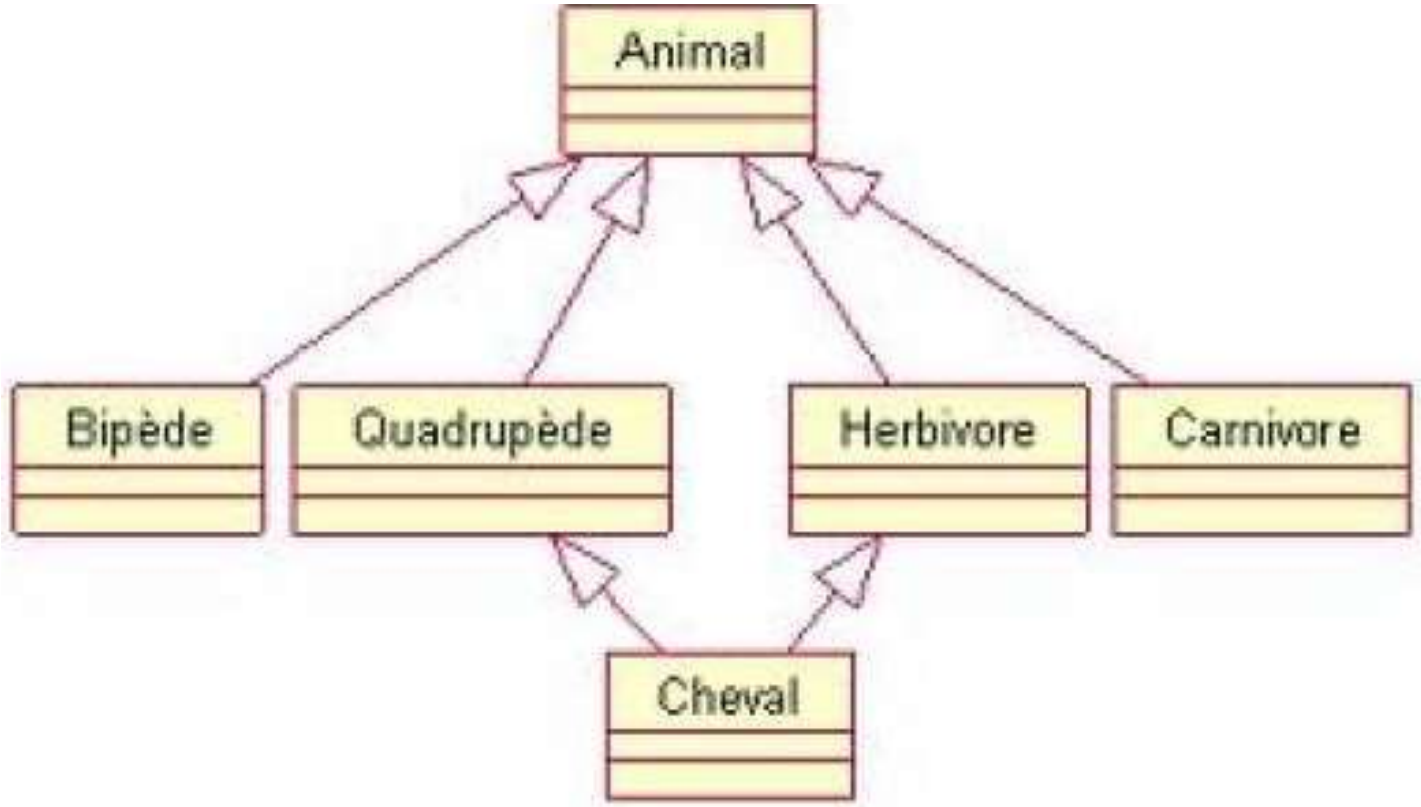


Exemple 1

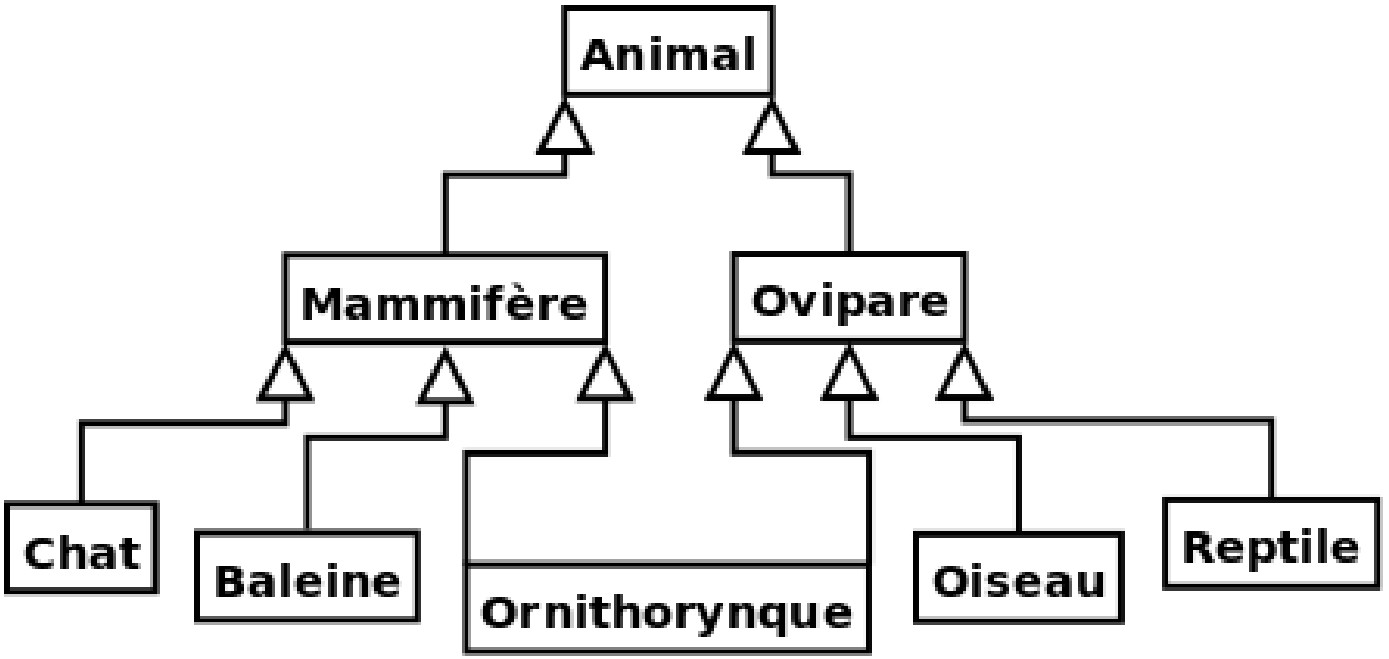

Moyen de transport
marque
modèle
vitesse
vitesse max
...
Voiture
numéro d'immatriculation
cylindrée
...
Bateau
tirant d'eau
nombre de voiles
...
Avion
altitude
altitude max
portée

Héritage multiple

- Une classe peut avoir plusieurs classes parents. On parle alors d'héritage multiple.



Exemple 2

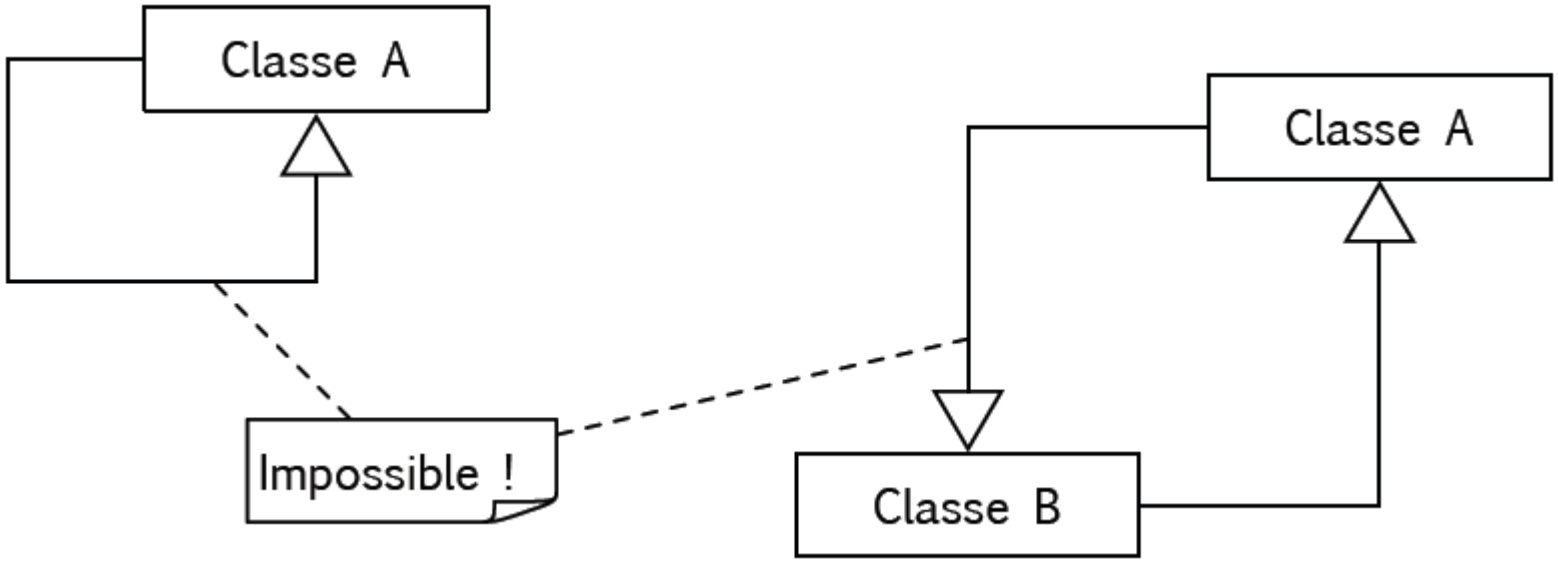

Animal
Mammifère
Ovipare
Chat
Baleine
Ornithorynque
Oiseau
Reptile

Les propriétés principales de l'héritage

- la classe enfant possède toutes les caractéristiques de ses classes parents, mais elle ne peut accéder aux caractéristiques privées de cette dernière ;
- une classe enfant peut redéfinir (même signature) une ou plusieurs méthodes de la classe parent. Sauf indication contraire, un objet utilise les opérations les plus spécialisées dans la hiérarchie des classes ;
- toutes les associations de la classe parent s'appliquent aux classes dérivées ;
- une instance d'une classe peut être utilisée partout où une instance de sa classe parent est attendue.
- une classe peut avoir plusieurs parents, on parle alors d'héritage multiple. Le langage C++ est un des langages objet permettant son implémentation effective, le langage Java ne le permet pas.

Généralisation non autorisé

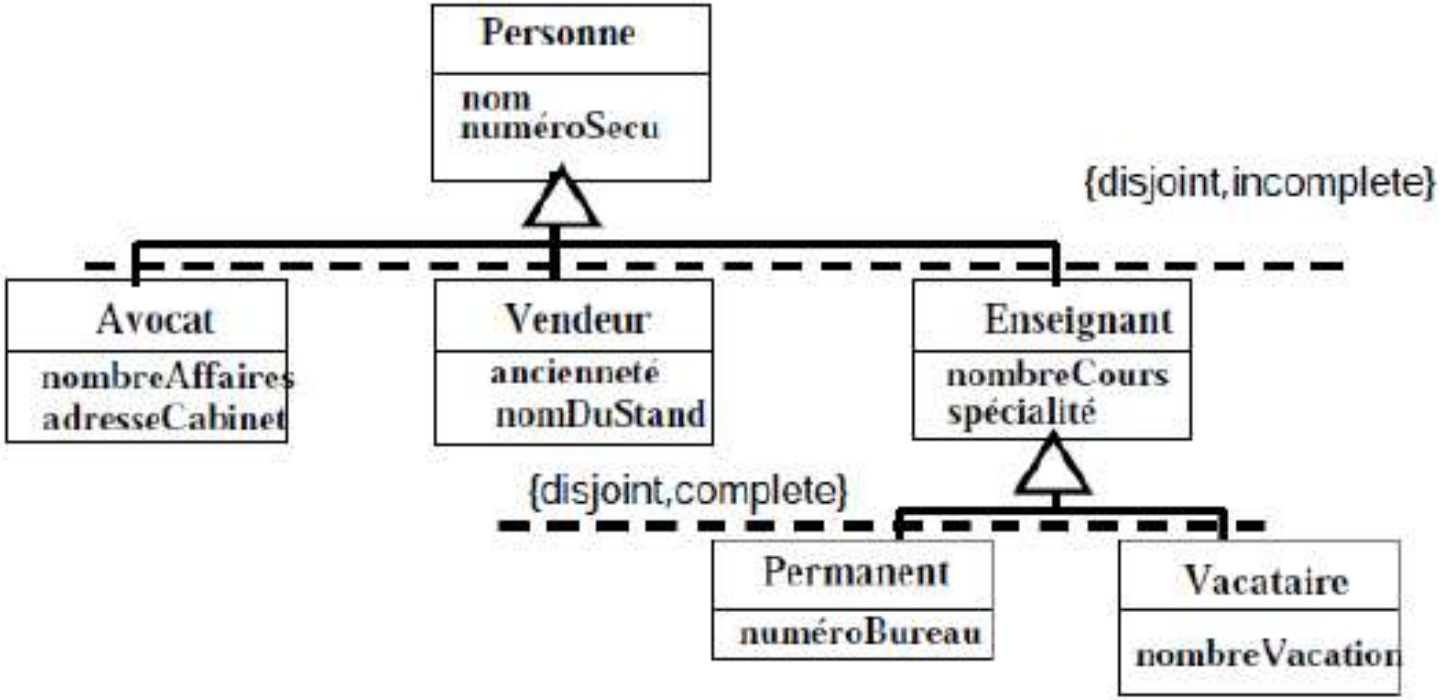
- Relation non-réflexive, non-symétrique !



Contraintes sur la relation d'héritage

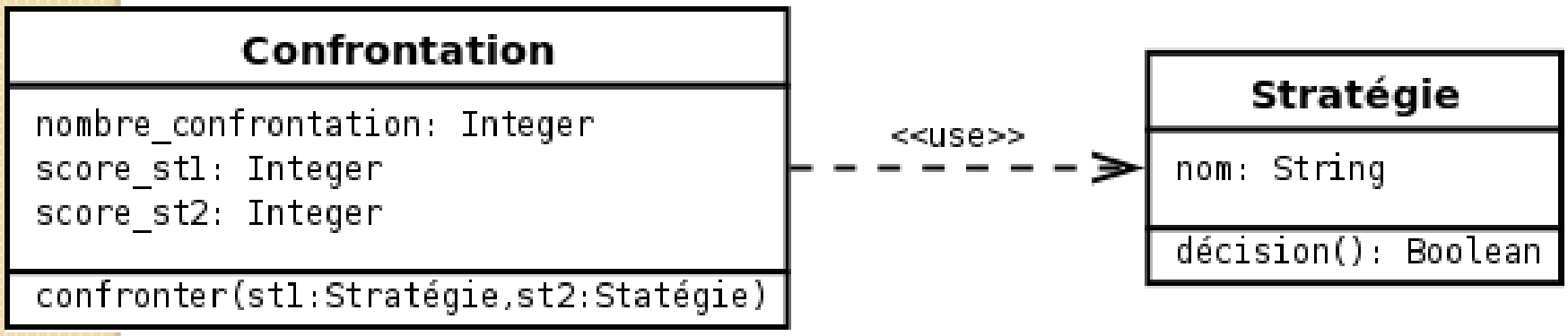
- Il existe quatre contraintes sur la relation d'héritage entre une surclasse et ses sous-classes :
  - {incomplete} : l'ensemble des sous-classes est incomplet et ne couvre pas la surclasse, c'est-à-dire que l'ensemble des instances des sous-classes est un sous-ensemble de l'ensemble des instances de la surclasse.
  - {complete} : l'ensemble des sous-classes est complet et couvre la surclasse.
  - {disjoint} : les sous-classes n'ont aucune instance en commun.
  - {overlapping} : les sous-classes peuvent avoir une ou plusieurs instances en commun.

Exemple


Personne
nom
numéroSecu
{disjoint,incomplete}
Avocat
nombreAffaires
adresseCabinet
Vendeur
ancienneté
nomDuStand
Enseignant
nombreCours
spécialité
{disjoint,complete}
Permanent
numéroBureau
Vacataire
nombreVacation

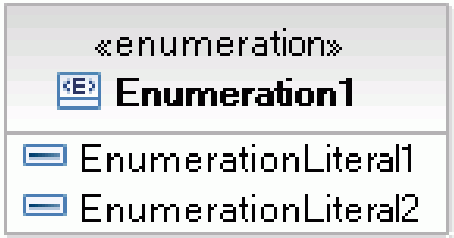
dépendance

- Une dépendance est une relation unidirectionnelle exprimant une dépendance sémantique entre des éléments du modèle.
- Elle indique que la modification de la cible peut impliquer une modification de la source.
- La dépendance est souvent stéréotypée pour mieux expliciter le lien sémantique entre les éléments du modèle.
- On utilise souvent une dépendance quand une classe en utilise une autre comme argument dans la signature d'une opération.
- Elle est représentée par un trait discontinu orienté.

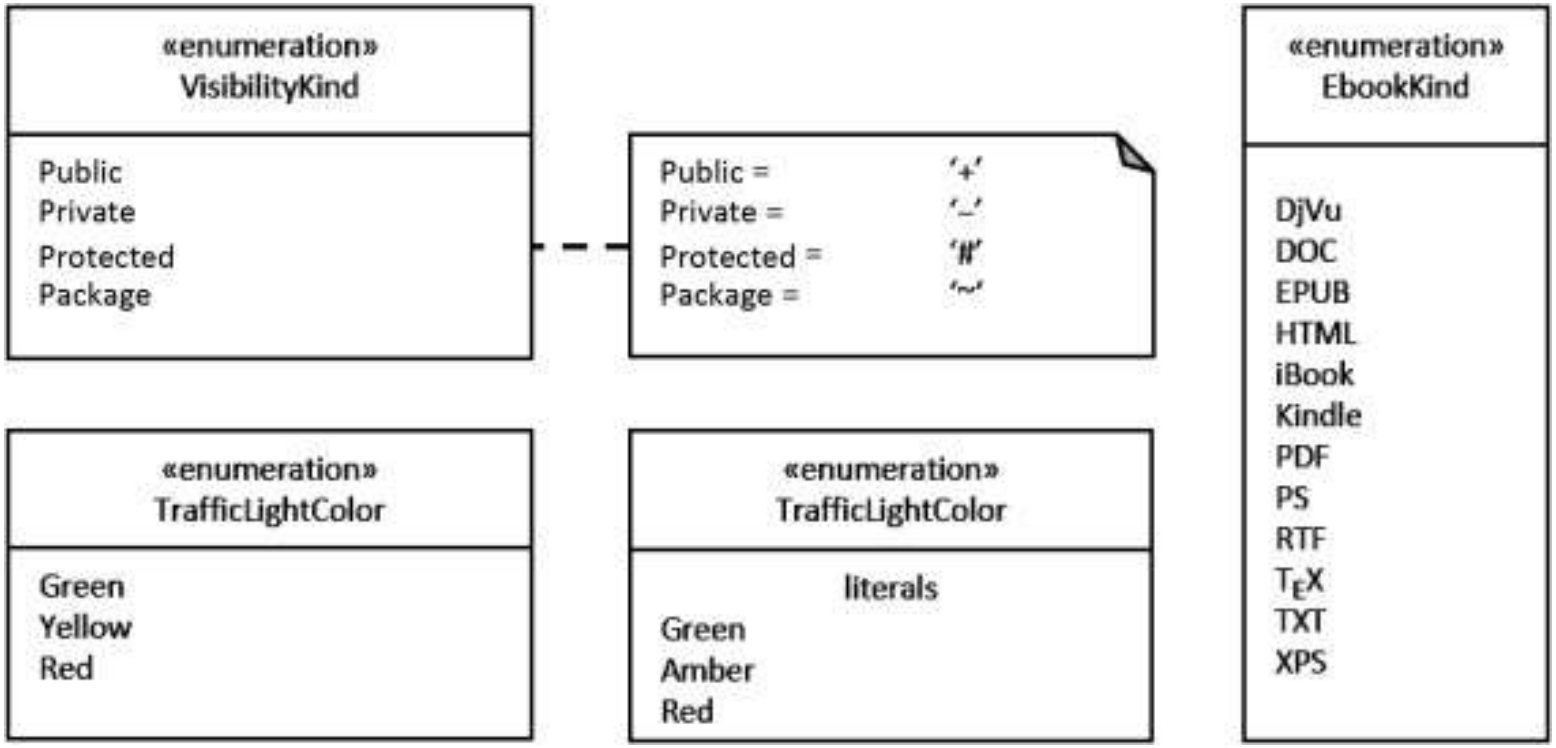


Enumérations

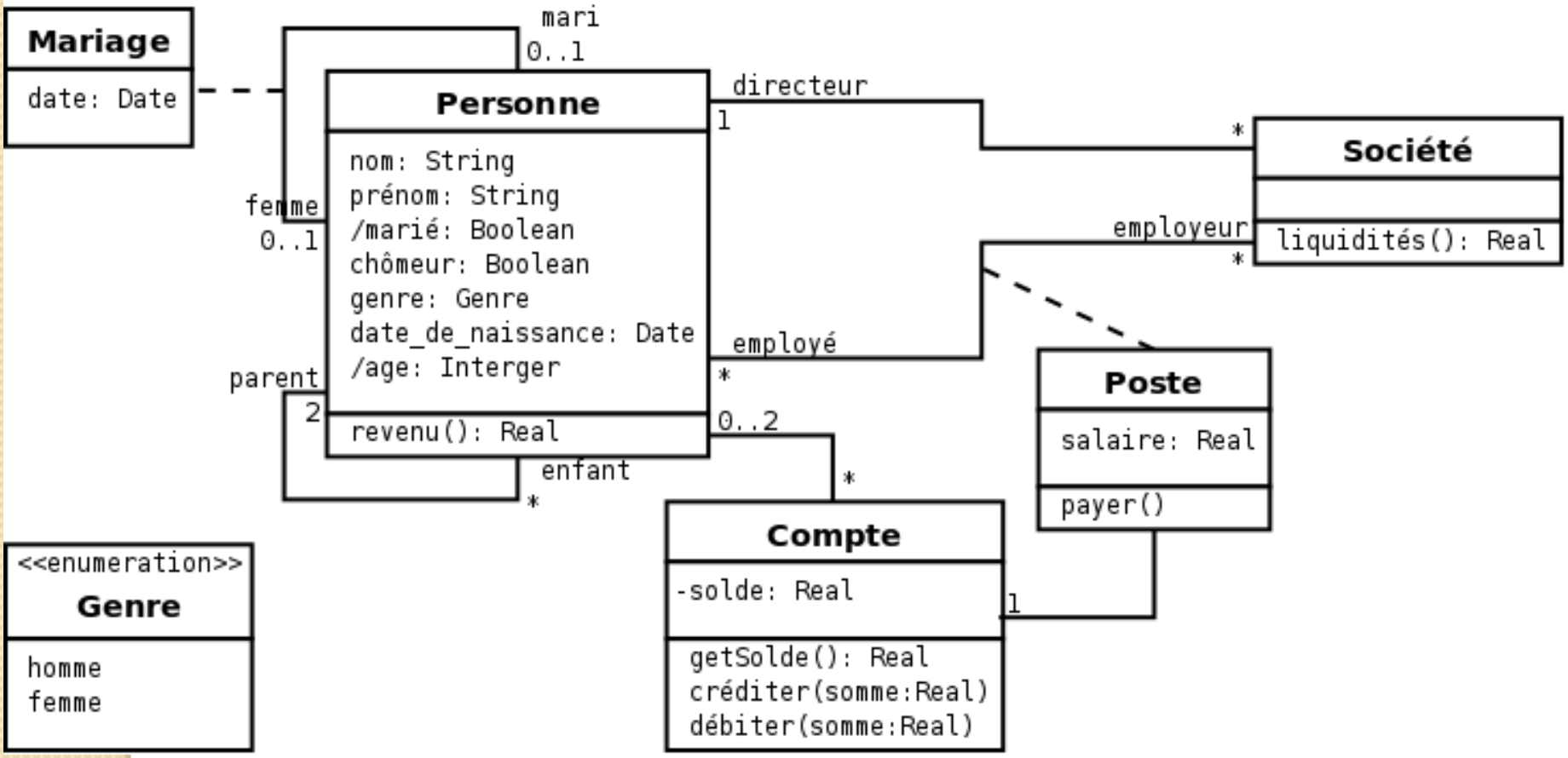
- les *énumérations* sont des éléments de modèle dans les diagrammes de classes qui représentent des types de données définis par l'utilisateur.
- Les énumérations contiennent des ensembles d'identificateurs nommés qui représentent les valeurs de l'énumération. Ces valeurs sont appelées littéraux d'énumération.



Exemple 1


«enumeration»
VisibilityKind
Public
Private
Protected
Package
Public = '+'
Private = '-'
Protected = '#'
Package = '~'
«enumeration»
EbookKind
DjVu
DOC
EPUB
HTML
iBook
Kindle
PDF
PS
RTF
TEX
TXT
XPS
«enumeration»
TrafficLightColor
Green
Yellow
Red
«enumeration»
TrafficLightColor
literals
Green
Amber
Red

Exemple


Mariage
date: Date
mari
0..1
Personne
nom: String
prénom: String
/marié: Boolean
chômeur: Boolean
genre: Genre
date_de_naissance: Date
/age: Interger
revenu(): Real
femme
0..1
parent
2
enfant
*
directeur
1
*
Société
liquidités(): Real
employeur
*
employé
*
Poste
salaire: Real
payer()
0..2
*
Compte
-solde: Real
getSolde(): Real
créditer(somme:Real)
débiter(somme:Real)
1
<<enumeration>>
Genre
homme
femme

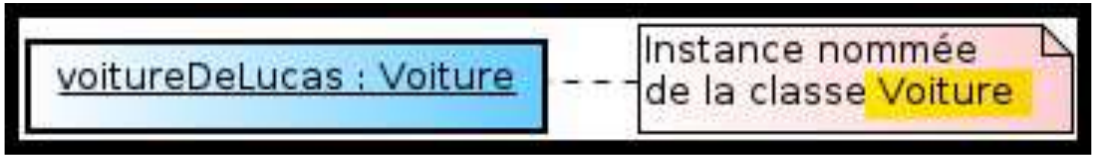
Un diagramme d'objets

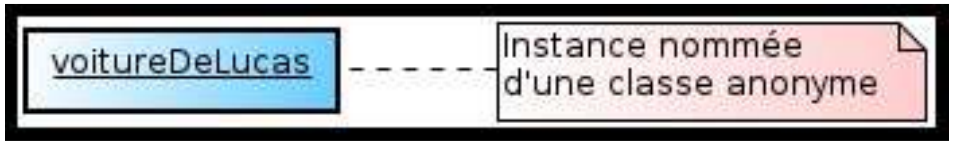
- Un diagramme d'objets représente
  - des objets (i.e. instances de classes) et
  - leurs liens (i.e. instances de relations) pour donner une vue figée de l'état d'un système à un instant donné.
- Un diagramme d'objets peut être utilisé pour :
  - illustrer le modèle de classes en montrant un exemple qui explique le modèle ;
  - préciser certains aspects du système en mettant en évidence des détails imperceptibles dans le diagramme de classes ;
  - exprimer une exception en modélisant des cas particuliers ou des connaissances non généralisables qui ne sont pas modélisés dans un diagramme de classe ;
  - prendre une image (snapshot) d'un système à un moment donné.
  - Un diagramme d'objets ne montre pas les interactions entre les objets
- Le diagramme de classes modélise les règles et le diagramme d'objets modélise des faits.

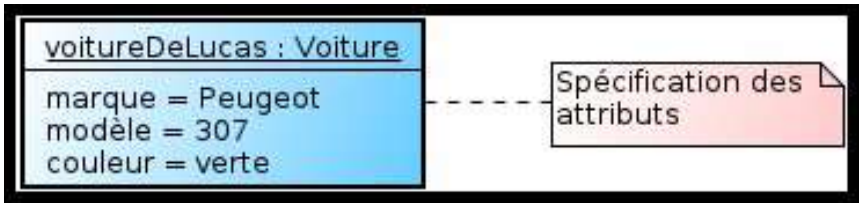
Représentation des objets

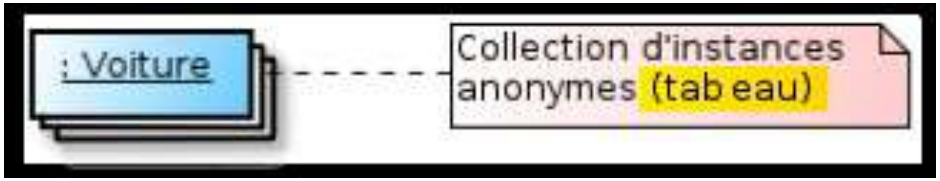
- Chaque objet est représenté dans un rectangle dans lequel figure le nom de l'objet (souligné) et éventuellement la valeur de un ou plusieurs de ses attributs.
- Comme pour la représentation d'une classe, la représentation d'un objet pourra être plus ou moins détaillée

Exemple commentés







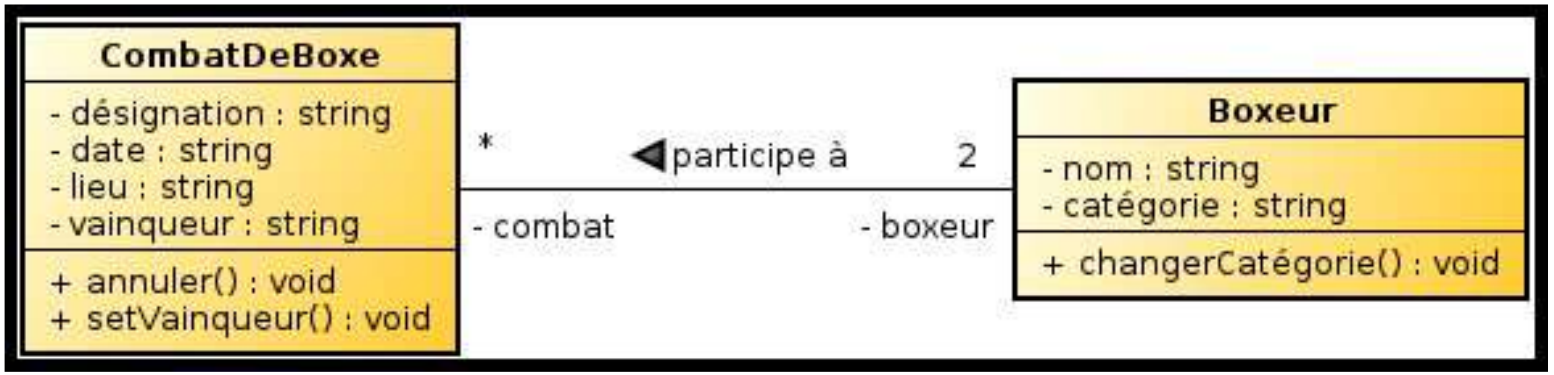


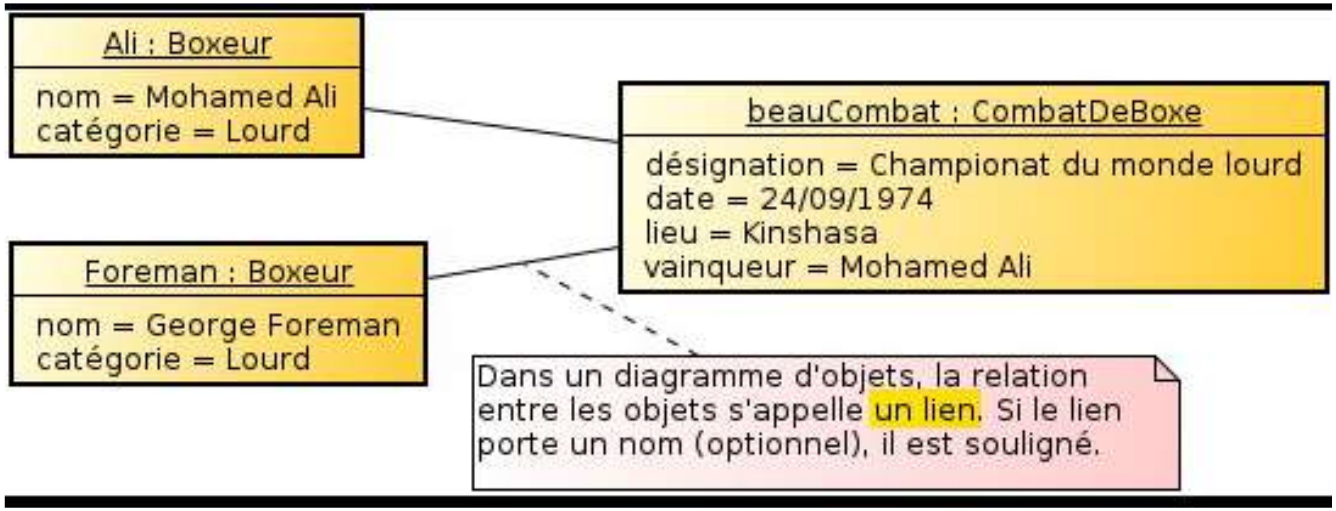
Les liens

- Les objets sont reliés par des instances d'associations : les liens.
- Un lien représente une relation entre objets a un instant donné.

- ATTENTION : la multiplicité des extrémités des liens est toujours de 1.
- Exemple : représentation de la structure générale d'une voiture

Exemple


CombatDeBoxe
- désignation : string
- date : string
- lieu : string
- vainqueur : string
+ annuler() : void
+ setVainqueur() : void
*
participe à
2
- combat
- boxeur
Boxeur
- nom : string
- catégorie : string
+ changerCatégorie() : void


Ali : Boxeur
nom = Mohamed Ali
catégorie = Lourd
Foreman : Boxeur
nom = George Foreman
catégorie = Lourd
beauCombat : CombatDeBoxe
désignation = Championat du monde lourd
date = 24/09/1974
lieu = Kinshasa
vainqueur = Mohamed Ali
Dans un diagramme d'objets, la relation entre les objets s'appelle un lien. Si le lien porte un nom (optionnel), il est souligné.

Relation de dépendance d'instanciation